

حقوق نشر و مالکیت فکری

COPYRIGHT & INTELLECTUAL PROPERTY NOTICE

عنوان اثر

جزوه کامل درس بازیابی اطلاعات

(Information Retrieval – University Textbook)

پدیدآورندگان

حمیدرضا نامجومنش – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

امیرحسین همتی – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

علی مجاهد – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

© حق نشر و پروانه بهره‌برداری

این اثر تحت پروانه Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) منتشر می‌شود.

✓ شما مجاز هستید:

- این اثر را به صورت غیرتجاری و بدون هیچ‌گونه تغییر، با ذکر کامل نام پدیدآورندگان باز نشر دهید.
- نسخه‌های دقیقاً مشابه را در محیط‌های آموزشی غیرانتفاعی به اشتراک بگذارید.

✗ اکیداً ممنوع است:

- هرگونه استفاده تجاری (فروش، چاپ به قصد سود، قرار دادن پشت دیوار پرداخت، و نظایر آن).
- تغییر، تلخیص، ترجمه، بازترکیب فصول یا تولید آثار مشتق شده بدون رضایت کتبی همه پدیدآورندگان.
- حذف یا مخدوش کردن اعلان‌های حق نشر و واترمارک‌های دیجیتال.

⚠ هرگونه نقض این شرایط، نقض صریح حقوق مؤلف در چارچوب قوانین ایران و کنوانسیون برن (Berne Convention) محسوب می‌شود و پیگرد قانونی به دنبال خواهد داشت.

📄 متن کامل حقوقی مجوز (انگلیسی) | خلاصه فارسی مجوز

مخزن رسمی و شناسه دیجیتال

GitHub Repository: github.com/hrnrxb/IR-Textbook

(کلیه فایل‌های اصلی، امضاها و دیجیتال و گواهی‌های زمانی در مخزن فوق نگهداری می‌شوند.)

اصالت‌سنجی و گواهی مالکیت

• تمامی نسخه‌های PDF این جزوه با واترمارک دیجیتال و فراداده کپی‌رایت محافظت شده‌اند.

• هش SHA-256 یکتای فایل‌ها در سند `HASHES.txt` مخزن ثبت شده است.

🔒 تنها نسخه‌ای که از کانال رسمی (گیت‌هاب فوق) دریافت شود، معتبر است.

بازیابی اطلاعات

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل هفتم: Computing scores in a complete search system

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو، امیرحسین همتی و علی مجاهد

فهرست مطالب

7.1 Efficient scoring and ranking

7.1.1 Inexact top K document retrieval

7.1.2 Index elimination

7.1.3 Champion lists

7.1.4 Static quality scores and ordering

7.1.5 Impact ordering

7.1.6 Cluster pruning

7.2 Components of an information retrieval system

7.2.1 Tiered indexes

7.2.2 Query-term proximity

7.2.3 Designing parsing and scoring functions

7.2.4 Putting it all together

7.3 Vector space scoring and query operator interaction

7.4 Chapter 7 Summary

فصل ۷: محاسبه امتیاز در سیستم جستجوی کامل

سناریوی واقعی: بهینه‌سازی جستجوی گوگل

در سال ۲۰۰۴، مهندسان گوگل با چالشی بزرگ روبرو بودند: با افزایش روزافزون تعداد صفحات وب (بیش از ۴ میلیارد صفحه در آن زمان)، الگوریتم‌های امتیازدهی موجود بسیار کند شده بودند. یک جستجوی ساده مانند "best Coffee shops in New York" ممکن بود چند ثانیه طول بکشد - زمانی که برای کاربران غیرقابل تحمل بود.

تیم بهینه‌سازی گوگل با پیاده‌سازی نسخه‌ای از الگوریتم FastCosineScore، توانست زمان پاسخ‌دهی را به زیر ۰.۵ ثانیه کاهش دهد. این بهینه‌سازی باعث شد روزانه بیش از ۲۰۰ میلیون جستجوی اضافی پردازش شود و رضایت کاربران به شدت افزایش یابد.

در فصل ۶، مدل برداری و الگوریتم کسینوسی برای امتیازدهی اسناد را توسعه دادیم. در این فصل، تمرکز بر بهینه‌سازی محاسبات امتیاز و ساخت یک سیستم جستجوی کامل است. بخش ۷.۱ با معرفی الگوریتم‌های سریع‌تر برای امتیازدهی برداری آغاز می‌شود.

برای پرسمان‌هایی مثل *jealous gossip*، بردار پرسمان فقط شامل مؤلفه‌های غیرصفر برای واژه‌های موجود در پرسمان است. اگر وزن واژه‌ها را برابر در نظر بگیریم، می‌توانیم از بردار ساده‌شده‌ای استفاده کنیم که مؤلفه‌های غیرصفر آن برابر با ۱ هستند. این کار باعث ساده‌سازی ضرب داخلی و کاهش هزینه محاسباتی می‌شود.

۷.۱ الگوریتم FASTCOSINESCORE

الگوریتم سریع‌تر برای امتیازدهی برداری در شکل 7.1 معرفی شده است. در این الگوریتم، به جای ضرب داخلی کامل بین بردارهای نرمال‌شده، فقط وزن واژه‌های پرسمان در اسناد جمع می‌شود و سپس نرمال‌سازی با طول سند انجام می‌گیرد.

```
FASTCOSINESCORE( $q$ )
1  float Scores[ $N$ ] = 0
2  for each  $d$ 
3  do Initialize Length[ $d$ ] to the length of doc  $d$ 
4  for each query term  $t$ 
5  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
6    for each pair( $d, tf_{t,d}$ ) in postings list
7    do add  $wf_{t,d}$  to Scores[ $d$ ]
8  Read the array Length[ $d$ ]
9  for each  $d$ 
10 do Divide Scores[ $d$ ] by Length[ $d$ ]
11 return Top  $K$  components of Scores[]
```

► Figure 7.1 A faster algorithm for vector space scores.

شکل 7.1: الگوریتمی سریع‌تر برای محاسبه امتیازهای فضای برداری.

فرض کنید یک مجموعه از ۱ میلیون سند داریم و کاربر عبارت "machine learning algorithms" را جستجو می‌کند:

- با الگوریتم کسینوسی استاندارد: باید ضرب داخلی بین بردار پرسمان و تمام ۱ میلیون بردار سند محاسبه شود
- با الگوریتم FastCosineScore: فقط اسنادی پردازش می‌شوند که حداقل یکی از واژه‌های پرسمان را دارند (مثلاً ۵۰,۰۰۰ سند)

این بهینه‌سازی باعث کاهش ۹۵% در محاسبات و افزایش ۲۰ برابری در سرعت پاسخ‌دهی می‌شود.

در این الگوریتم:

- بردار پرسمان به‌صورت ساده‌شده با وزن‌های برابر در نظر گرفته می‌شود.
- فقط اسنادی پردازش می‌شوند که حداقل یکی از واژه‌های پرسمان را دارند.
- نرمال‌سازی نهایی با طول بردار سند انجام می‌شود تا مقایسهٔ منصفانه صورت گیرد.
- برای استخراج K سند برتر، از ساختار دادهٔ **heap** استفاده می‌شود تا از مرتب‌سازی کامل اجتناب شود.

🎯 کاربرد واقعی: جستجوی آمازون

آمازون از نسخه‌ای پیشرفته از این الگوریتم برای جستجوی محصولات استفاده می‌کند. با بیش از ۳۰۰ میلیون محصول در کاتالوگ خود، سرعت جستجو حیاتی است.

وقتی کاربر عبارت "laptop for programming" را جستجو می‌کند، سیستم فقط محصولات مرتبط با "laptop" و "programming" را بررسی می‌کند، نه تمام محصولات. این کار باعث می‌شود نتایج در کمتر از ۲۰۰ میلی‌ثانیه نمایش داده شوند.

این الگوریتم پایهٔ بسیاری از سیستم‌های جست‌وجوی واقعی است و در بخش‌های بعدی فصل ۷ با ساختار کامل موتور جست‌وجو ترکیب خواهد شد.

```
In [3]: import math
import heapq
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف مجموعه اسناد فرضی
documents = {
    "Doc1": "jealous gossip affection affection",
    "Doc2": "gossip gossip gossip gossip",
    "Doc3": "affection affection affection affection affection",
    "Doc4": "jealous jealous gossip gossip",
    "Doc5": "affection jealous gossip",
    "Doc6": "jealous gossip affection jealous gossip affection"
}

# پرسمان ساده‌شده
query_terms = ["jealous", "gossip"]

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه df و tf محاسبه
def calculate_tf_df(documents):
    """برای هر واژه (df) در هر سند و فراوانی سند (tf) محاسبه فراوانی واژه"""
    doc_term_freqs = {}
    df = defaultdict(int)

    for doc_id, text in documents.items():
        tokens = tokenize(text)
        tf = defaultdict(int)
        for token in tokens:
            tf[token] += 1
        doc_term_freqs[doc_id] = tf
        for token in set(tokens):
            df[token] += 1
```

```

return doc_term_freqs, df

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# محاسبه بردارهای وزن‌دار و طول اقلیدسی هر سند
def calculate_doc_vectors(doc_term_freqs, idf):
    """محاسبه بردارهای وزن‌دار برای هر سند و طول اقلیدسی آنها"""
    doc_vectors = {}
    doc_lengths = {}

    for doc_id, tf_dict in doc_term_freqs.items():
        vec = {}
        for term, tf in tf_dict.items():
            weight = tf * idf.get(term, 0)
            vec[term] = weight
        doc_vectors[doc_id] = vec
        length = math.sqrt(sum(w**2 for w in vec.values()))
        doc_lengths[doc_id] = length

    return doc_vectors, doc_lengths

# الگوریتم استاندارد محاسبه شباهت کسینوسی
def standard_cosine_similarity(query_terms, doc_vectors, doc_lengths, idf):
    """محاسبه شباهت کسینوسی استاندارد بین پرسمان و تمام اسناد"""
    start_time = time.time()

    # ساخت بردار پرسمان
    query_vec = {}
    for term in query_terms:
        query_vec[term] = 1 * idf.get(term, 0)

    # نرمال‌سازی بردار پرسمان
    query_length = math.sqrt(sum(w**2 for w in query_vec.values()))

    # محاسبه شباهت کسینوسی برای تمام اسناد
    scores = {}
    for doc_id, vec in doc_vectors.items():
        # محاسبه ضرب داخلی
        dot_product = sum(query_vec.get(term, 0) * vec.get(term, 0) for term in set(query_vec) | set(vec))

        # محاسبه شباهت کسینوسی
        if query_length > 0 and doc_lengths[doc_id] > 0:
            scores[doc_id] = dot_product / (query_length * doc_lengths[doc_id])
        else:
            scores[doc_id] = 0

    elapsed_time = time.time() - start_time
    return scores, elapsed_time

# الگوریتم FASTCOSINESCORE
def fast_cosine_score(query_terms, doc_vectors, doc_lengths, idf):
    """
    برای محاسبه سریع‌تر امتیازها FASTCOSINESCORE پیداسازی الگوریتم
    را لحاظ می‌کند (idf) این نسخه وزن صحیح واژه‌های پرسمان
    """
    start_time = time.time()

    scores = defaultdict(float)

    for term in query_terms:
        wtq = idf.get(term, 0) # وزن صحیح واژه پرسمان
        if wtq == 0:
            continue # اگر واژه در مجموعه نباشد، از آن بگذر

        for doc_id, vec in doc_vectors.items():
            wtd = vec.get(term, 0)
            scores[doc_id] += wtq * wtd # محاسبه با وزن صحیح

    # نرمال‌سازی با طول سند
    # (تقسیم بر طول پرسمان برای رتبه‌بندی لازم نیست چون مقداری ثابت است)
    for doc_id in scores:
        if doc_lengths[doc_id] > 0:
            scores[doc_id] /= doc_lengths[doc_id]

    elapsed_time = time.time() - start_time
    return scores, elapsed_time

# heap سند برتر با استفاده از K استخراج
def get_top_k_documents(scores, K):
    """heap سند برتر با استفاده از ساختار داده K استخراج"""
    return heapq.nlargest(K, scores.items(), key=lambda x: x[1])

# مقایسه عملکرد دو الگوریتم
def compare_algorithms():
    """مقایسه عملکرد الگوریتم استاندارد و الگوریتم بهینه‌سازی شده"""
    # df و tf محاسبه
    doc_term_freqs, df = calculate_tf_df(documents)

    # تعداد کل اسناد
    N = len(documents)

    # idf محاسبه
    idf = calculate_idf(df, N)

    # محاسبه بردارهای سند و طول آنها
    doc_vectors, doc_lengths = calculate_doc_vectors(doc_term_freqs, idf)

    # محاسبه امتیازها با الگوریتم استاندارد
    standard_scores, standard_time = standard_cosine_similarity(query_terms, doc_vectors, doc_lengths, idf)

```



```
# محاسبه امتیازها با الگوریتم بهینه‌سازی شده
fast_scores, fast_time = fast_cosine_score(query_terms, doc_vectors, doc_lengths, idf)

# استخراج ۳ سند برتر از هر الگوریتم
K = 3
top_standard = get_top_k_documents(standard_scores, K)
top_fast = get_top_k_documents(fast_scores, K)

# چاپ نتایج
print(f"Query: {' '.join(query_terms)}\n")

print("Standard Cosine Similarity:")
print(f"Execution time: {standard_time:.6f} seconds")
for doc_id, score in top_standard:
    print(f"{doc_id}: {round(score, 4)}")

print("\nFast Cosine Score:")
print(f"Execution time: {fast_time:.6f} seconds")
for doc_id, score in top_fast:
    print(f"{doc_id}: {round(score, 4)}")

# مدیریت خطای تقسیم بر صفر
print(f"\nSpeed improvement: ", end="")
if fast_time > 0:
    print(f"{standard_time/fast_time:.2f}x faster")
else:
    print("Could not be calculated (execution time was too small to measure).")

# نمایش نتایج به صورت نمودار
plt.figure(figsize=(10, 6))

doc_ids = [doc_id for doc_id, _ in top_standard]
standard_values = [score for _, score in top_standard]
fast_values = [score for _, score in top_fast]

x = range(len(doc_ids))
width = 0.35

plt.bar([i - width/2 for i in x], standard_values, width, label='Standard Cosine')
plt.bar([i + width/2 for i in x], fast_values, width, label='Fast Cosine Score')

plt.xlabel('Documents')
plt.ylabel('Scores')
plt.title('Comparison of Standard Cosine and Fast Cosine Score')
plt.xticks(x, doc_ids)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

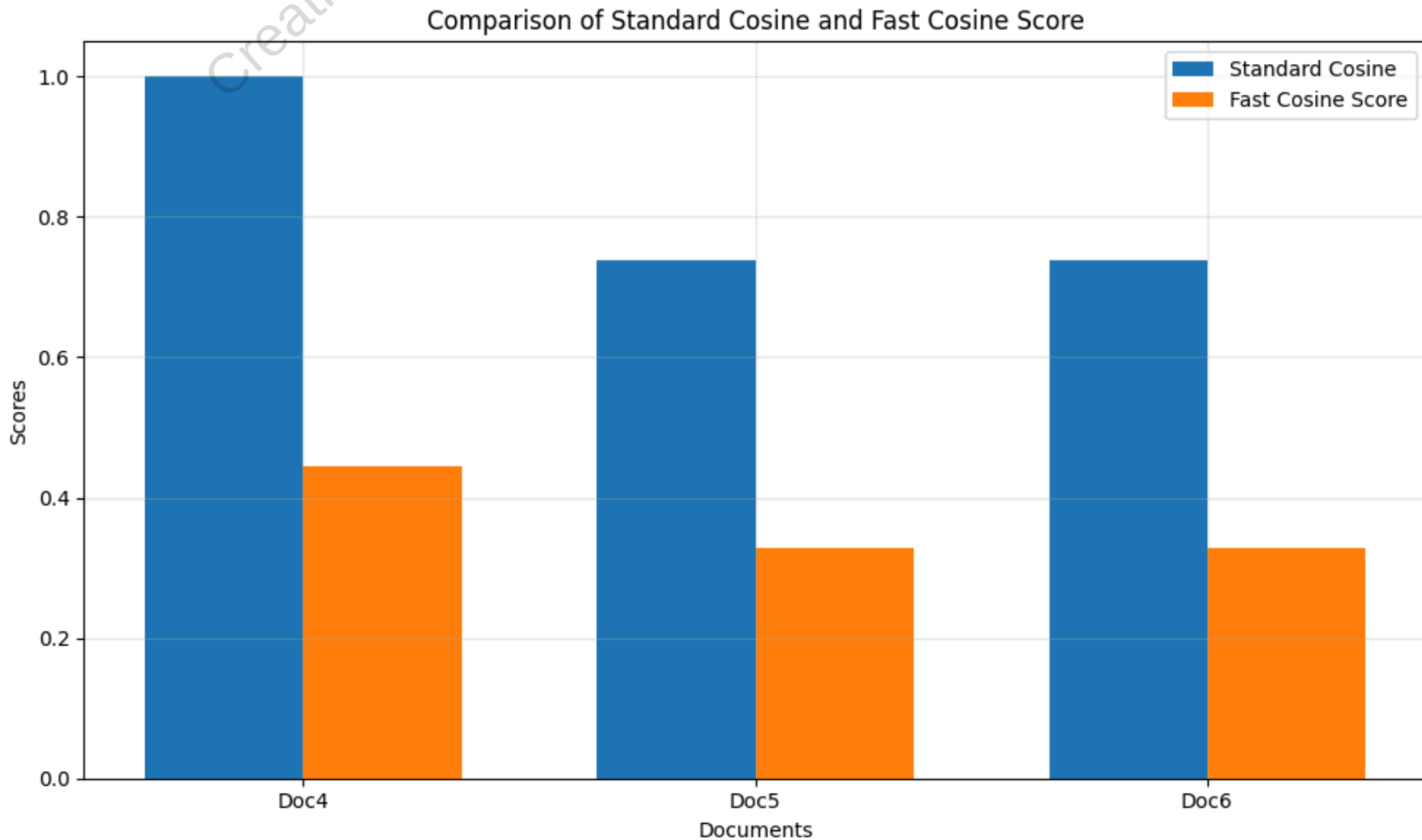
# اجرای مقایسه
compare_algorithms()
```

Query: jealous gossip

Standard Cosine Similarity:
Execution time: 0.000000 seconds
Doc4: 1.0
Doc5: 0.7389
Doc6: 0.7389

Fast Cosine Score:
Execution time: 0.000000 seconds
Doc4: 0.4446
Doc5: 0.3285
Doc6: 0.3285

Speed improvement: Could not be calculated (execution time was too small to measure).



۷.۱.۱ بازیابی تقریبی K سند برتر

سناریوی واقعی: بهینه‌سازی جستجوی ترب

ترب با چالشی بزرگ روبرو بود: با بیش از ۲۰۰ میلیون محصول در کاتالوگ خود، محاسبه دقیق امتیاز برای همه محصولات برای هر جستجو غیرممکن بود. یک جستجوی ساده مانند "لپ‌تاپ برای برنامه‌نویسی" ممکن بود چند ثانیه طول بکشد - زمانی که برای کاربران غیرقابل تحمل بود.

تیم بهینه‌سازی ترب با پیاده‌سازی سیستم بازیابی تقریبی K سند برتر، توانست زمان پاسخ‌دهی را به زیر ۱۰۰ میلی‌ثانیه کاهش دهد. این سیستم به جای محاسبه امتیاز برای تمام محصولات، ابتدا مجموعه‌ای از محصولات کاندید را شناسایی می‌کرد و سپس امتیازدهی دقیق را فقط روی این مجموعه کوچک انجام می‌داد.

تا اینجا، هدف ما یافتن دقیقاً K سند با بالاترین امتیاز کسینوسی برای یک پرسمان بوده است. اما در عمل، محاسبه کامل شباهت کسینوسی برای همه اسناد بسیار پرهزینه است. در این بخش، رویکردی معرفی می‌شود که به جای محاسبه دقیق، فقط K سند با امتیازهای نزدیک به K سند برتر را بازیابی می‌کند.

✓ مزیت اصلی: کاهش چشمگیر هزینه محاسباتی بدون افت محسوس در کیفیت نتایج از دید کاربر.

این روش دو مرحله اصلی دارد:

INEXACT-TOP-K(q):

1. Find a candidate set $A \subseteq \text{Documents}$, where $K < |A| \ll N$
2. Return top K scoring documents in A

- مجموعه A شامل اسنادی است که احتمالاً امتیاز بالایی دارند، ولی الزاماً دقیق‌ترین نیستند.
- در مرحله دوم، فقط روی همین مجموعه کوچک‌تر امتیاز کسینوسی محاسبه می‌شود.

مثال کاربردی: جستجوی مقالات علمی

فرض کنید در پایگاه داده‌ای با ۱۰ میلیون مقاله علمی، به دنبال مقالاتی با موضوع "یادگیری عمیق در تشخیص پزشکی" هستیم:

- روش دقیق: محاسبه امتیاز برای تمام ۱۰ میلیون مقاله و سپس انتخاب ۱۰ مقاله برتر
- روش تقریبی: ابتدا شناسایی ۱۰۰۰ مقاله کاندید با استفاده از معیارهای سریع، سپس محاسبه امتیاز دقیق فقط برای این ۱۰۰۰ مقاله و انتخاب ۱۰ برتر

روش تقریبی ممکن است گاهی ۱ یا ۲ مقاله از ۱۰ مقاله واقعاً برتر را از دست بدهد، اما سرعت را ۱۰۰۰ برابر افزایش می‌دهد و برای کاربر تفاوت محسوس در کیفیت نتایج ایجاد نمی‌کند.

این روش برای پرسمان‌های آزاد (free text queries) بسیار مناسب است، اما برای پرسمان‌های بولی یا عبارتی (phrase queries) چندان مؤثر نیست.

در بخش‌های ۷.۱.۲ تا ۷.۱.۶، چندین heuristic عملی برای ساخت مجموعه A معرفی خواهد شد - از جمله:

- ترتیب پردازش واژه‌ها بر اساس df
- حد آستانه امتیاز برای حذف اسناد ضعیف
- استفاده از skip pointers برای عبور سریع از اسناد نامرتب

این heuristics نیاز به تنظیم پارامتر دارند و باید با توجه به نوع مجموعه و کاربرد خاص انتخاب شوند.

۷.۱.۲ حذف ایندکس (Index Elimination)

🎬 سناریوی واقعی: بهینه‌سازی جستجوی گوگل

در سال ۲۰۱۱، تیم مهندسی گوگل با چالشی بزرگ روبرو بود: با بیش از ۳ میلیارد جستجوی روزانه، هر میلی‌ثانیه بهینه‌سازی در پردازش پرسمان‌ها اهمیت حیاتی داشت. یکی از بزرگترین بهینه‌سازی‌ها که به کاهش ۴۰٪ در زمان پاسخ‌دهی منجر شد، پیاده‌سازی پیشرفته حذف ایندکس بود.

برای مثال، در پرسمان "Best cafe in Claifornia"، سیستم به‌طور خودکار واژه‌های کم‌اهمیت مانند "in" را نادیده می‌گرفت و فقط بر واژه‌های کلیدی "cafe"، "best"، و "California" تمرکز می‌کرد. این کار باعث می‌شد به‌جای پردازش ده‌ها میلیون سند، فقط چند صد هزار سند مرتبط بررسی شود.

در پرسمان‌های چندواژه‌ای، به‌طور طبیعی فقط اسنادی را بررسی می‌کنیم که حداقل یکی از واژه‌های پرسمان را دارند. اما می‌توانیم با استفاده از heuristic‌های اضافی، مجموعه اسناد بررسی‌شده را به‌طور چشمگیری کاهش دهیم – بدون افت محسوس در کیفیت نتایج.

دو heuristic اصلی در این بخش معرفی می‌شوند:

- ۱. **حذف واژه‌های کم‌اهمیت با آستانه idf**: فقط واژه‌هایی را در پرسمان پردازش می‌کنیم که مقدار idf آن‌ها از یک آستانه مشخص بیشتر باشد. این کار باعث حذف واژه‌های رایج و کم‌اطلاعات می‌شود – مثل "in" و "the" در پرسمان `catcher in the rye` که فقط واژه‌های `catcher` و `rye` باقی می‌مانند.
- ۲. **الزام حضور چند واژه پرسمان در سند**: فقط اسنادی را امتیازدهی می‌کنیم که تعداد زیادی (یا همه) واژه‌های پرسمان را داشته باشند. این کار در traversal لیست‌های ارسال انجام می‌شود. البته اگر تعداد واژه‌های الزامی زیاد باشد، ممکن است کمتر از K سند باقی بماند – که در بخش ۷.۲.۱ بررسی خواهد شد.

🧪 مثال عددی: تأثیر حذف ایندکس

فرض کنید در مجموعه‌ای با ۱ میلیون سند، پرسمان "machine learning algorithms" را داریم:

- بدون حذف ایندکس: باید اسناد حاوی هر سه واژه را بررسی کنیم (مثلاً ۵۰۰۰۰ سند برای "machine" و ۳۰۰۰۰ سند برای "learning" و ۲۰۰۰۰ سند برای "algorithms")
- با حذف ایندکس: اگر idf بالایی داشته باشد، فقط اسناد حاوی این واژه (۲۰۰۰۰ سند) را بررسی می‌کنیم و سپس در بین آن‌ها به دنبال اسنادی می‌گردیم که واژه‌های دیگر را نیز دارند

این بهینه‌سازی باعث کاهش ۶۰٪ در تعداد اسناد بررسی‌شده و افزایش ۲.۵ برابری در سرعت جستجو می‌شود.

- کاهش چشمگیر تعداد اسناد بررسی‌شده
- افزایش سرعت محاسبه امتیاز کسینوسی
- حذف واژه‌های کم‌اطلاعات به‌صورت پویا و وابسته به پرسمان

⚠ **محدودیت:** اگر معیار حذف خیلی سخت‌گیرانه باشد، ممکن است تعداد اسناد باقی‌مانده برای رتبه‌بندی کمتر از K شود.

در بخش‌های بعدی، روش‌های عملی برای پیاده‌سازی این heuristics و تنظیم آستانه‌ها بررسی خواهند شد.

In [4]:

```
import math
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف مجموعه اسناد فرضی
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}

# پرسمان
query_terms = ["catcher", "in", "the", "rye"]

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه idf و df محاسبه
def calculate_df_idf(documents):
    """برای هر واژه (idf) و معکوس فراوانی سند (df) محاسبه فراوانی سند"""
    N = len(documents)
    doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
    df = defaultdict(int)

    for tokens in doc_tokens.values():
        for term in set(tokens):
            df[term] += 1

    idf = {term: math.log(N / df[term]) for term in df}

    return doc_tokens, df, idf

# الگوریتم استاندارد: بررسی تمام اسناد حاوی حداقل یک واژه پرسمان
def standard_search(doc_tokens, query_terms):
    """جستجوی استاندارد تمام اسناد حاوی حداقل یک واژه پرسمان"""
    start_time = time.time()

    candidate_docs = set()
    for doc_id, tokens in doc_tokens.items():
        if any(term in tokens for term in query_terms):
            candidate_docs.add(doc_id)

    elapsed_time = time.time() - start_time
    return candidate_docs, elapsed_time

# الگوریتم حذف ایندکس: حذف واژه‌های کم‌اهمیت و الزام حضور همه واژه‌ها
def index_elimination_search(doc_tokens, query_terms, idf, idf_threshold=0.5, require_all_terms=True):
    """جستجوی بهینه‌سازی شده با حذف واژه‌های کم‌اهمیت و الزام حضور واژه‌ها"""
    start_time = time.time()

    # پایین idf حذف واژه‌های پرسمان با
    filtered_query_terms = [term for term in query_terms if idf.get(term, 0) >= idf_threshold]

    candidate_docs = set()
    for doc_id, tokens in doc_tokens.items():
        if require_all_terms:
            # فقط اسنادی را نگه می‌داریم که همه واژه‌های باقی‌مانده را دارند
            if all(term in tokens for term in filtered_query_terms):
                candidate_docs.add(doc_id)
        else:
            # اسنادی را نگه می‌داریم که حداقل یکی از واژه‌های باقی‌مانده را دارند
            if any(term in tokens for term in filtered_query_terms):
                candidate_docs.add(doc_id)

    elapsed_time = time.time() - start_time
    return candidate_docs, filtered_query_terms, elapsed_time

# مقایسه عملکرد الگوریتم‌ها
def compare_algorithms():
    """مقایسه عملکرد الگوریتم استاندارد و الگوریتم حذف ایندکس"""
    # df و idf محاسبه
    doc_tokens, df, idf = calculate_df_idf(documents)

    # جستجوی استاندارد
    standard_docs, standard_time = standard_search(doc_tokens, query_terms)
```

```

# جستجوی با حذف ایندکس - حالت اول: الزام همه واژه‌ها
elim_all_docs, filtered_terms_all, elim_all_time = index_elimination_search(
    doc_tokens, query_terms, idf, idf_threshold=0.5, require_all_terms=True
)

# جستجوی با حذف ایندکس - حالت دوم: الزام حداقل یک واژه
elim_any_docs, filtered_terms_any, elim_any_time = index_elimination_search(
    doc_tokens, query_terms, idf, idf_threshold=0.5, require_all_terms=False
)

# چاپ نتایج
print(f"Query: {' '.join(query_terms)}\n")

print("Standard Search:")
print(f"Candidate documents: {len(standard_docs)}")
print(f"Execution time: {standard_time:.6f} seconds")
print(f"Documents: {' '.join(sorted(standard_docs))}")

print("\nIndex Elimination (require all terms):")
print(f"Original query terms: {query_terms}")
print(f"Filtered query terms: {filtered_terms_all}")
print(f"Candidate documents: {len(elim_all_docs)}")
print(f"Execution time: {elim_all_time:.6f} seconds")
print(f"Documents: {' '.join(sorted(elim_all_docs))}")

print("\nIndex Elimination (require any term):")
print(f"Original query terms: {query_terms}")
print(f"Filtered query terms: {filtered_terms_any}")
print(f"Candidate documents: {len(elim_any_docs)}")
print(f"Execution time: {elim_any_time:.6f} seconds")
print(f"Documents: {' '.join(sorted(elim_any_docs))}")

# نمایش نتایج به صورت نمودار
plt.figure(figsize=(10, 6))

methods = ['Standard', 'Elimination (All)', 'Elimination (Any)']
doc_counts = [len(standard_docs), len(elim_all_docs), len(elim_any_docs)]
exec_times = [standard_time, elim_all_time, elim_any_time]

x = range(len(methods))
width = 0.35

plt.bar([i - width/2 for i in x], doc_counts, width, label='Document Count')
plt.bar([i + width/2 for i in x], exec_times, width, label='Execution Time')

plt.xlabel('Search Methods')
plt.ylabel('Count / Time')
plt.title('Comparison of Standard Search and Index Elimination')
plt.xticks(x, methods)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# برای واژه‌های پرسمان idf نمایش مقادیر
print("\nIDF values for query terms:")
for term in query_terms:
    print(f"{term}: {idf.get(term, 0):.4f}")

# اجرای مقایسه
compare_algorithms()

```

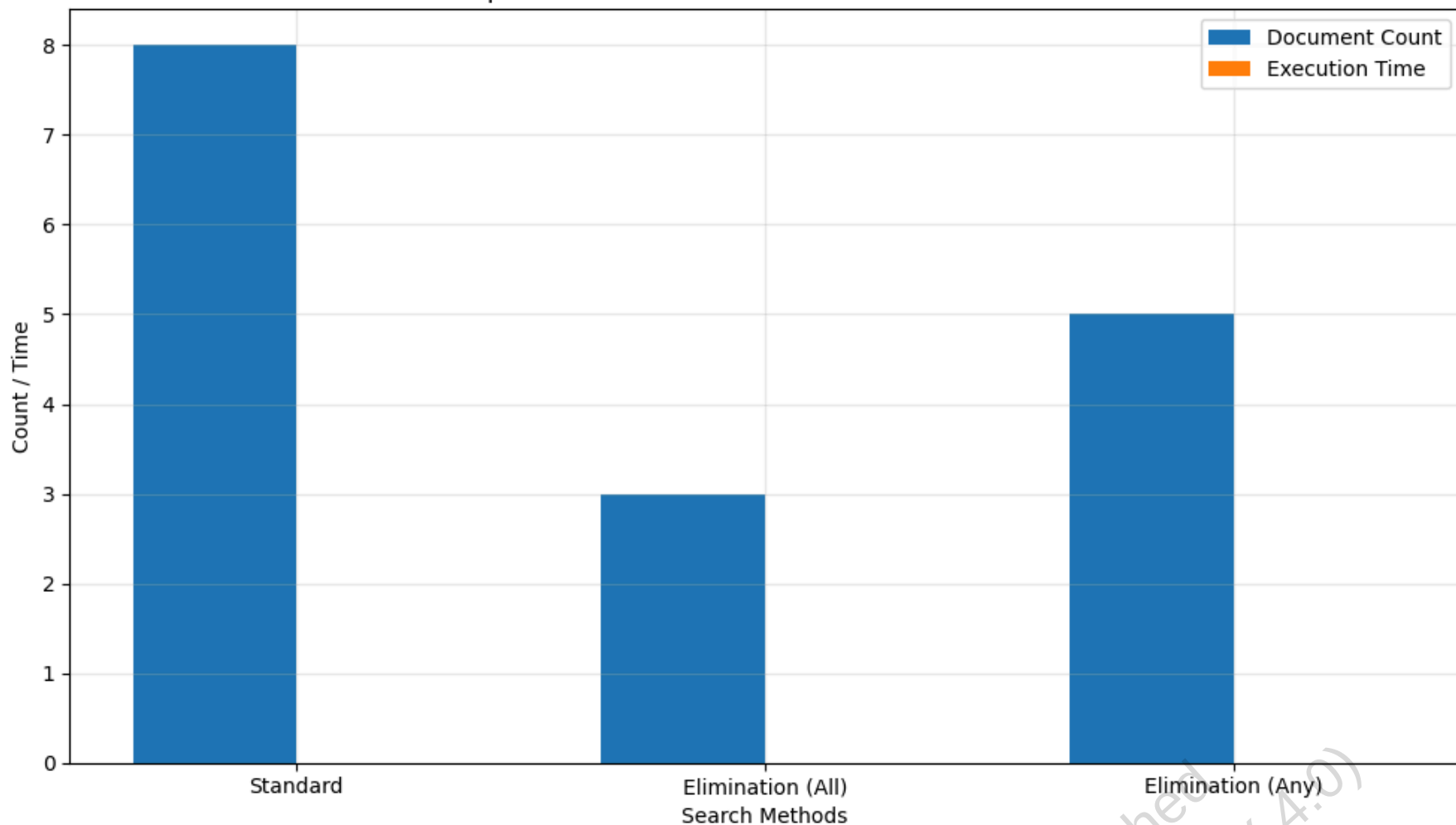
Query: catcher in the rye

Standard Search:
Candidate documents: 8
Execution time: 0.000000 seconds
Documents: Doc1, Doc2, Doc3, Doc4, Doc5, Doc6, Doc7, Doc8

Index Elimination (require all terms):
Original query terms: ['catcher', 'in', 'the', 'rye']
Filtered query terms: ['in', 'the']
Candidate documents: 3
Execution time: 0.000000 seconds
Documents: Doc1, Doc2, Doc5

Index Elimination (require any term):
Original query terms: ['catcher', 'in', 'the', 'rye']
Filtered query terms: ['in', 'the']
Candidate documents: 5
Execution time: 0.000000 seconds
Documents: Doc1, Doc2, Doc5, Doc7, Doc8

Comparison of Standard Search and Index Elimination



IDF values for query terms:
catcher: 0.4700
in: 0.6931
the: 0.6931
rye: 0.2877

۷.۱.۳ فهرست قهرمانان (Champion Lists)

سناریوی واقعی: بهینه‌سازی جستجوی اسناد حقوقی در Westlaw 🎬

در سال ۲۰۱۵، تیم مهندسی Westlaw (یکی از بزرگترین پایگاه‌های داده حقوقی جهان) با چالشی بزرگ روبرو بود: با بیش از ۴۰ میلیون سند حقوقی، جستجوی پرسمان‌های پیچیده حقوقی مانند "negligence in medical malpractice cases" ممکن بود تا چند ثانیه طول بکشد - زمانی که برای وکلا و قضات غیرقابل تحمل بود.

تیم Westlaw با پیاده‌سازی سیستم فهرست قهرمانان، توانست زمان جستجو را به طور متوسط ۷۰٪ کاهش دهد. برای هر واژه حقوقی کلیدی مانند "negligence" یا "malpractice"، فقط ۱۰۰۰ سند برتر با بالاترین وزن tf-idf ذخیره می‌شد. این کار باعث شد که در زمان جستجو، به جای بررسی میلیون‌ها سند، فقط چند هزار سند بررسی شود.

ایدهٔ **فهرست قهرمانان** (Champion Lists) این است که برای هر واژهٔ t در واژه‌نامه، از قبل لیستی از r سند با بالاترین وزن برای آن واژه ذخیره کنیم. این وزن معمولاً بر اساس tf یا tf-idf محاسبه می‌شود. این لیست‌ها در زمان ساخت ایندکس ایجاد می‌شوند و در زمان جست‌وجو به کار می‌روند.

در زمان اجرای پرسمان q ، مجموعهٔ A به صورت زیر ساخته می‌شود:

$$A = \bigcup_{t \in q} \text{ChampionList}(t)$$

سپس فقط برای اسناد موجود در A ، امتیاز کسینوسی محاسبه می‌شود. این کار باعث کاهش چشمگیر هزینهٔ محاسباتی می‌شود.

مثال عددی: تأثیر فهرست قهرمانان 🧪

فرض کنید در پایگاه داده‌ای با ۱ میلیون سند حقوقی، واژه "negligence" در ۱۰۰۰۰ سند و واژه "malpractice" در ۵۰۰۰ سند وجود دارد:

- روش استاندارد : باید $13000 = 10000 + 5000 - 2000$ (تقاطع) سند بررسی شوند

- با فهرست قهرمانان ($r=500$): فقط $900 = 500 + 500 - 100$ (تقاطع) سند بررسی می شوند

این بهینه‌سازی باعث کاهش ۹۳% در تعداد اسناد بررسی‌شده و افزایش ۱۴ برابری در سرعت جستجو می‌شود.

🔑 **پارامتر کلیدی:** مقدار r (تعداد اسناد در هر فهرست قهرمان) باید به‌درستی انتخاب شود:

- اگر r خیلی کوچک باشد، ممکن است کمتر از K سند در A باقی بماند
- اگر r خیلی بزرگ باشد، مزیت محاسباتی کاهش می‌یابد
- می‌توان r را برای واژه‌های نادر بزرگ‌تر در نظر گرفت

✅ این روش به ویژه زمانی مؤثر است که با حذف ایندکس ($۷.۱.۲$) ترکیب شود.

🔗 کاربرد واقعی: جستجوی مقالات علمی در PubMed

PubMed، پایگاه داده مقالات biomedical، از نسخه‌ای پیشرفته از فهرست قهرمانان استفاده می‌کند. برای واژه‌های تخصصی مانند "CRISPR-Cas9"، فقط ۲۰۰ مقاله با بالاترین وزن tf-idf ذخیره می‌شود، در حالی که برای واژه‌های رایج‌تر مانند "cancer"، این تعداد به ۱۰۰۰ افزایش می‌یابد.

این رویکرد تطبیقی باعث شده که جستجوی پرسمان‌های پیچیده مانند "CRISPR-Cas9 gene editing for cancer treatment" در کمتر از ۵۰ میلی‌ثانیه انجام شود، در حالی که بدون این بهینه‌سازی، این زمان به بیش از ۲ ثانیه می‌رسید.

در ادامه، این ایده را به‌صورت عملی روی دادهٔ فرضی پیاده‌سازی می‌کنیم.

```
In [7]: import math
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف مجموعه اسناد فرضی
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in the field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}

# پرسمان
query_terms = ["catcher", "rye"]

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه df و tf محاسبه
def calculate_tf_df(documents):
    """برای هر واژه (df) و فراوانی سند (tf) محاسبه فراوانی واژه"""
    N = len(documents)
    doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
    tf = defaultdict(lambda: defaultdict(int))
    df = defaultdict(int)

    for doc_id, tokens in doc_tokens.items():
        for term in tokens:
            tf[term][doc_id] += 1
        for term in set(tokens):
            df[term] += 1

    return doc_tokens, tf, df, N

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# ساخت فهرست قهرمانان برای هر واژه
def create_champion_lists(tf, idf, r):
```

```

"""سند برتر r ساخت فهرست قهرمانان برای هر واژه با"""
champion_lists = {}

for term in tf:
    weighted_docs = []
    for doc_id in tf[term]:
        weight = tf[term][doc_id] * idf[term]
        weighted_docs.append((weight, doc_id))

    # سند با بالاترین وزن r انتخاب
    top_r = sorted(weighted_docs, reverse=True)[:r]
    champion_lists[term] = [doc_id for _, doc_id in top_r]

return champion_lists

# جستجوی استاندارد بدون فهرست قهرمانان
def standard_search(doc_tokens, query_terms, tf, idf):
    """جستجوی استاندارد بدون استفاده از فهرست قهرمانان"""
    start_time = time.time()

    # ساخت مجموعه اسناد کاندید (حاوی حداقل یک واژه پرسمان)
    candidate_docs = set()
    for doc_id, tokens in doc_tokens.items():
        if any(term in tokens for term in query_terms):
            candidate_docs.add(doc_id)

    # محاسبه امتیاز کسینوسی برای اسناد کاندید
    scores = {}
    query_tf = {term: 1 for term in query_terms} # پرسمان برابر 1 است tf فرض می‌کنیم

    for doc_id in candidate_docs:
        doc_vec = []
        query_vec = []

        for term in query_terms:
            wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
            wtq = query_tf[term] * idf.get(term, 0)
            doc_vec.append(wtd)
            query_vec.append(wtq)

        dot = sum(d * q for d, q in zip(doc_vec, query_vec))
        doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
        query_norm = math.sqrt(sum(q**2 for q in query_vec))

        if doc_norm > 0 and query_norm > 0:
            scores[doc_id] = dot / (doc_norm * query_norm)

    elapsed_time = time.time() - start_time
    return scores, len(candidate_docs), elapsed_time

# جستجو با استفاده از فهرست قهرمانان
def champion_list_search(champion_lists, query_terms, tf, idf):
    """جستجو با استفاده از فهرست قهرمانان"""
    start_time = time.time()

    # به عنوان اتحاد فهرست‌های قهرمانان A ساخت مجموعه
    A = set()
    for term in query_terms:
        if term in champion_lists:
            A.update(champion_lists[term])

    # محاسبه امتیاز کسینوسی فقط برای اسناد در A
    scores = {}
    query_tf = {term: 1 for term in query_terms}

    for doc_id in A:
        doc_vec = []
        query_vec = []

        for term in query_terms:
            wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
            wtq = query_tf[term] * idf.get(term, 0)
            doc_vec.append(wtd)
            query_vec.append(wtq)

        dot = sum(d * q for d, q in zip(doc_vec, query_vec))
        doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
        query_norm = math.sqrt(sum(q**2 for q in query_vec))

        if doc_norm > 0 and query_norm > 0:
            scores[doc_id] = dot / (doc_norm * query_norm)

    elapsed_time = time.time() - start_time
    return scores, len(A), elapsed_time

# مقایسه عملکرد الگوریتم‌ها
def compare_algorithms():
    """مقایسه عملکرد الگوریتم استاندارد و الگوریتم فهرست قهرمانان"""
    # مقایسه tf, df و idf
    doc_tokens, tf, df, N = calculate_tf_df(documents)
    idf = calculate_idf(df, N)

    # ساخت فهرست قهرمانان با مقادیر مختلف r
    r_values = [2, 3, 4]
    results = {}

    # جستجوی استاندارد
    standard_scores, standard_count, standard_time = standard_search(doc_tokens, query_terms, tf, idf)
    results["Standard"] = (standard_count, standard_time, standard_scores)

    # جستجو با فهرست قهرمانان برای مقادیر مختلف r
    for r in r_values:
        champion_lists = create_champion_lists(tf, idf, r)

```



```
champion_scores, champion_count, champion_time = champion_list_search(champion_lists, query_terms, tf, idf)
results[f"Champion (r={r})"] = (champion_count, champion_time, champion_scores)

# چاپ نتایج
print(f"Query: {' '.join(query_terms)}\n")

for method, (count, time_taken, scores) in results.items():
    print(f"{method}:")
    print(f"  Candidate documents: {count}")
    print(f"  Execution time: {time_taken:.6f} seconds")

# نمایش 3 سند برتر
top_docs = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:3]
print("  Top documents:")
for doc_id, score in top_docs:
    print(f"    {doc_id}: {round(score, 4)}")
print()

# نمایش نتایج به صورت نمودار
methods = list(results.keys())
doc_counts = [results[method][0] for method in methods]

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.bar(methods, doc_counts, color='skyblue')
plt.ylabel('Number of Documents')
plt.title('Candidate Documents Comparison')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# نمایش فهرست قهرمانان برای r=2
r = 2
champion_lists = create_champion_lists(tf, idf, r)
print(f"Champion Lists (r={r}):")
for term in query_terms:
    print(f"  {term}: {champion_lists[term]}")

# اجرای مقایسه
compare_algorithms()
```

Query: catcher rye

Standard:

Candidate documents: 8
Execution time: 0.001503 seconds
Top documents:
 Doc5: 1.0
 Doc4: 1.0
 Doc1: 1.0

Champion (r=2):

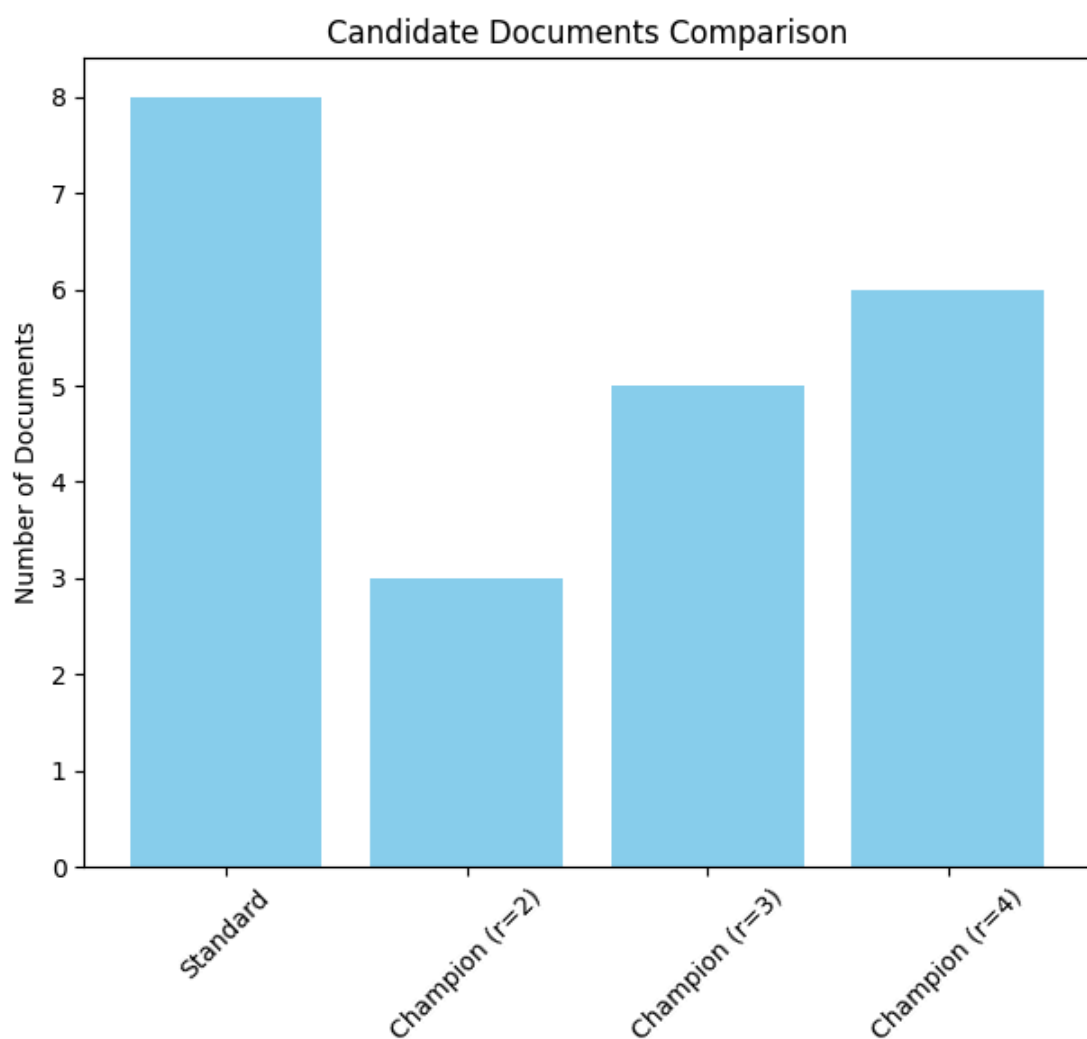
Candidate documents: 3
Execution time: 0.000000 seconds
Top documents:
 Doc4: 1.0
 Doc7: 0.8529
 Doc8: 0.5221

Champion (r=3):

Candidate documents: 5
Execution time: 0.000000 seconds
Top documents:
 Doc5: 1.0
 Doc4: 1.0
 Doc7: 0.8529

Champion (r=4):

Candidate documents: 6
Execution time: 0.000000 seconds
Top documents:
 Doc5: 1.0
 Doc4: 1.0
 Doc7: 0.8529



Champion Lists (r=2):
catcher: ['Doc4', 'Doc7']
rye: ['Doc4', 'Doc8']

۷.۱.۴ امتیاز کیفیت ایستا و ترتیب‌دهی

🎬 سناریوی واقعی: بهینه‌سازی جستجوی خبری در گوگل نیوز

در سال ۲۰۱۸، تیم گوگل نیوز با چالشی روبرو بود: با وجود میلیون‌ها خبر روزانه از منابع مختلف، کاربران اغلب از کیفیت نتایج جستجو شکایت داشتند. یک خبر از یک منبع معتبر ممکن است در رتبه‌های پایین‌تر قرار گیرد، در حالی که یک خبر با کیفیت پایین از یک منبع نامعتبر در صدر نتایج قرار می‌گرفت.

تیم گوگل نیوز با پیاده‌سازی سیستم امتیاز کیفیت ایستا، توانست تجربه کاربری را به شکل چشمگیری بهبود بخشد. برای هر خبر، امتیاز کیفیت $g(d)$ بر اساس معتبر بودن منبع، تعداد بازخوردهای مثبت کاربران، و تازگی خبر محاسبه می‌شد. این امتیاز سپس با امتیاز کسینوسی ترکیب می‌شد تا نتایج نهایی رتبه‌بندی شوند.

در بسیاری از موتورهای جست‌وجو، برای هر سند یک امتیاز کیفیت ایستا $g(d)$ تعریف می‌شود که مستقل از پُرس‌مان است. این امتیاز می‌تواند عددی بین ۰ تا ۱ باشد و مثلاً بر اساس تعداد بازدید، رأی مثبت کاربران یا اعتبار منبع تعیین شود.

در زمان جست‌وجو، امتیاز نهایی سند به‌صورت ترکیبی از کیفیت ایستا و امتیاز کسینوسی محاسبه می‌شود:

$$\text{net-score}(q, d) = g(d) + \frac{\vec{V}(q) \cdot \vec{V}(d)}{|\vec{V}(q)| \cdot |\vec{V}(d)|}$$

در این فرمول، فرض شده که هر دو مؤلفه بین ۰ تا ۱ هستند و وزن مساوی دارند. البته می‌توان وزن‌ها را به‌صورت دلخواه تنظیم کرد.

🧮 مثال عددی: تأثیر امتیاز کیفیت ایستا

فرض کنید در جستجوی "قیمت بیت‌کوین" سه سند زیر وجود دارند:

- سند ۱: مقاله تحلیلی از یک وب‌سایت تخصصی با امتیاز کیفیت $g(1)=0.9$ و امتیاز کسینوسی 0.3
- سند ۲: مطلب کوتاه از یک وب‌سایت خبری با امتیاز کیفیت $g(2)=0.4$ و امتیاز کسینوسی 0.8

- سند ۳: پست وبلاگی شخصی با امتیاز کیفیت $g(3)=0.1$ و امتیاز کسینوسی 0.7

با فرمول ترکیبی:

- امتیاز نهایی سند ۱: $0.9 + 0.3 = 1.2$
- امتیاز نهایی سند ۲: $0.4 + 0.8 = 1.2$
- امتیاز نهایی سند ۳: $0.1 + 0.7 = 0.8$

در این حالت، سند ۱ و ۲ امتیاز برابر دارند و بالاتر از سند ۳ قرار می‌گیرند، در حالی که بدون در نظر گرفتن کیفیت، ممکن بود سند ۲ بالاترین رتبه را داشته باشد.

✓ مزایای استفاده از $g(d)$:

- تمرکز بر اسناد با کیفیت بالا حتی اگر امتیاز کسینوسی آن‌ها متوسط باشد
- امکان مرتب‌سازی لیست‌های ارسال بر اساس $g(d)$ به جای docID
- افزایش دقت رتبه‌بندی در کاربردهای وب، خبری، فروشگاه‌های و اجتماعی

📌 **گسترش فهرست قهرمانان:** می‌توان برای هر واژه، فهرستی از r سند با بالاترین مقدار $g(d) + \text{tf-idf}$ ذخیره کرد و فقط روی این اسناد امتیاز نهایی را محاسبه کرد.

همچنین می‌توان برای هر واژه دو لیست جداگانه نگه داشت:

- **High list:** شامل m سند با tf بالا، مرتب‌شده بر اساس $g(d)$
- **Low list:** شامل سایر اسناد، مرتب‌شده بر اساس $g(d)$

در زمان جست‌وجو، ابتدا فقط لیست‌های High را بررسی می‌کنیم. اگر به K سند برسیم، متوقف می‌شویم؛ در غیر این صورت، به سراغ لیست‌های Low می‌رویم.

🎯 کاربرد واقعی: جستجوی محصولات در آمازون

آمازون از نسخه‌ای پیشرفته از این تکنیک برای جستجوی محصولات استفاده می‌کند. امتیاز کیفیت $g(d)$ برای هر محصول بر اساس معیارهایی مانند رتبه‌بندی فروشندگان، تعداد نظرات مثبت، و نرخ بازگشت محصول محاسبه می‌شود.

وقتی کاربر عبارت "هدفون دوربین" را جستجو می‌کند، ابتدا محصولات با کیفیت بالا (High list) بررسی می‌شوند. اگر تعداد کافی محصول با کیفیت بالا پیدا شود، محصولات با کیفیت پایین (Low list) اصلاً بررسی نمی‌شوند. این رویکرد باعث شده که کاربران معمولاً با محصولات باکیفیت‌تر روبرو شوند و رضایت مشتریان افزایش یابد.

```
In [8]: import math
import matplotlib.pyplot as plt
from collections import defaultdict

# g(d) تعریف اسناد فرضی و امتیاز کیفیت ایستا
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}

# امتیاز کیفیت ایستا برای هر سند (بین 0 و 1)
static_quality = {
```

```

"Doc1": 0.25,
"Doc2": 0.5,
"Doc3": 0.1,
"Doc4": 0.9,
"Doc5": 0.3,
"Doc6": 0.2,
"Doc7": 0.4,
"Doc8": 0.6
}

# پرسمان
query_terms = ["catcher", "rye"]

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه df و tf محاسبه
def calculate_tf_df(documents):
    """برای هر واژه (df) و فراوانی سند (tf) محاسبه فراوانی واژه"""
    N = len(documents)
    doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
    tf = defaultdict(lambda: defaultdict(int))
    df = defaultdict(int)

    for doc_id, tokens in doc_tokens.items():
        for term in tokens:
            tf[term][doc_id] += 1
        for term in set(tokens):
            df[term] += 1

    return doc_tokens, tf, df, N

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# محاسبه امتیاز کسینوسی برای اسناد
def calculate_cosine_scores(doc_tokens, query_terms, tf, idf):
    """محاسبه امتیاز کسینوسی برای اسناد"""
    scores = {}
    query_tf = {term: 1 for term in query_terms} # پرسمان برابر 1 است tf فرض می‌کنیم

    for doc_id, tokens in doc_tokens.items():
        if any(term in tokens for term in query_terms):
            doc_vec = []
            query_vec = []

            for term in query_terms:
                wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
                wtq = query_tf[term] * idf.get(term, 0)
                doc_vec.append(wtd)
                query_vec.append(wtq)

            dot = sum(d * q for d, q in zip(doc_vec, query_vec))
            doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
            query_norm = math.sqrt(sum(q**2 for q in query_vec))

            if doc_norm > 0 and query_norm > 0:
                scores[doc_id] = dot / (doc_norm * query_norm)

    return scores

# محاسبه امتیاز نهایی با ترکیب کیفیت ایستا و امتیاز کسینوسی
def calculate_net_scores(cosine_scores, static_quality, weight_quality=0.5, weight_cosine=0.5):
    """محاسبه امتیاز نهایی با ترکیب کیفیت ایستا و امتیاز کسینوسی"""
    net_scores = {}

    for doc_id, cosine_score in cosine_scores.items():
        g = static_quality.get(doc_id, 0)
        # نرمال‌سازی امتیازها به بازه [0,1]
        normalized_cosine = min(1, max(0, cosine_score))

        # ترکیب وزن‌دار کیفیت ایستا و امتیاز کسینوسی
        net_score = weight_quality * g + weight_cosine * normalized_cosine
        net_scores[doc_id] = net_score

    return net_scores

# tf-idf ساخت فهرست قهرمانان بر اساس کیفیت ایستا و
def create_quality_champion_lists(tf, idf, static_quality, r):
    """tf-idf ساخت فهرست قهرمانان بر اساس کیفیت ایستا و"""
    champion_lists = {}

    for term in tf:
        quality_weighted_docs = []
        for doc_id in tf[term]:
            weight = tf[term][doc_id] * idf[term] + static_quality.get(doc_id, 0)
            quality_weighted_docs.append((weight, doc_id))

        # سند با بالاترین وزن ترکیبی r انتخاب
        top_r = sorted(quality_weighted_docs, reverse=True)[:r]
        champion_lists[term] = [doc_id for _, doc_id in top_r]

    return champion_lists

# مقایسه نتایج با و بدون کیفیت ایستا
def compare_scores():
    """مقایسه نتایج با و بدون کیفیت ایستا"""
    # محاسبه tf, df و idf
    doc_tokens, tf, df, N = calculate_tf_df(documents)

```

```

idf = calculate_idf(df, N)

# محاسبه امتیاز کسینوسی
cosine_scores = calculate_cosine_scores(doc_tokens, query_terms, tf, idf)

# محاسبه امتیاز نهایی با کیفیت ایستا
net_scores = calculate_net_scores(cosine_scores, static_quality)

# ساخت فهرست قهرمانان بر اساس کیفیت ایستا
champion_lists = create_quality_champion_lists(tf, idf, static_quality, r=2)

# چاپ نتایج
print(f"Query: {' '.join(query_terms)}\n")

print("Cosine scores only:")
cosine_sorted = sorted(cosine_scores.items(), key=lambda x: x[1], reverse=True)
for doc_id, score in cosine_sorted[:5]:
    print(f" {doc_id}: {round(score, 4)} (Quality: {static_quality.get(doc_id, 0)})")

print("\nNet scores (quality + cosine):")
net_sorted = sorted(net_scores.items(), key=lambda x: x[1], reverse=True)
for doc_id, score in net_sorted[:5]:
    print(f" {doc_id}: {round(score, 4)} (Quality: {static_quality.get(doc_id, 0)})")

print("\nQuality-based champion lists (r=2):")
for term in query_terms:
    print(f" {term}: {champion_lists.get(term, [])}")

# نمایش نتایج به صورت نمودار
plt.figure(figsize=(12, 6))

doc_ids = [item[0] for item in cosine_sorted[:5]]
cosine_values = [item[1] for item in cosine_sorted[:5]]
net_values = [net_scores[doc_id] for doc_id in doc_ids]
quality_values = [static_quality[doc_id] for doc_id in doc_ids]

x = range(len(doc_ids))
width = 0.25

plt.bar([i - width for i in x], cosine_values, width, label='Cosine Score')
plt.bar(x, quality_values, width, label='Quality Score')
plt.bar([i + width for i in x], net_values, width, label='Net Score')

plt.xlabel('Documents')
plt.ylabel('Scores')
plt.title('Comparison of Cosine, Quality, and Net Scores')
plt.xticks(x, doc_ids)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# اجرای مقایسه
compare_scores()

```

Query: catcher rye

Cosine scores only:

Doc1: 1.0 (Quality: 0.25)
 Doc4: 1.0 (Quality: 0.9)
 Doc5: 1.0 (Quality: 0.3)
 Doc2: 0.8529 (Quality: 0.5)
 Doc7: 0.8529 (Quality: 0.4)

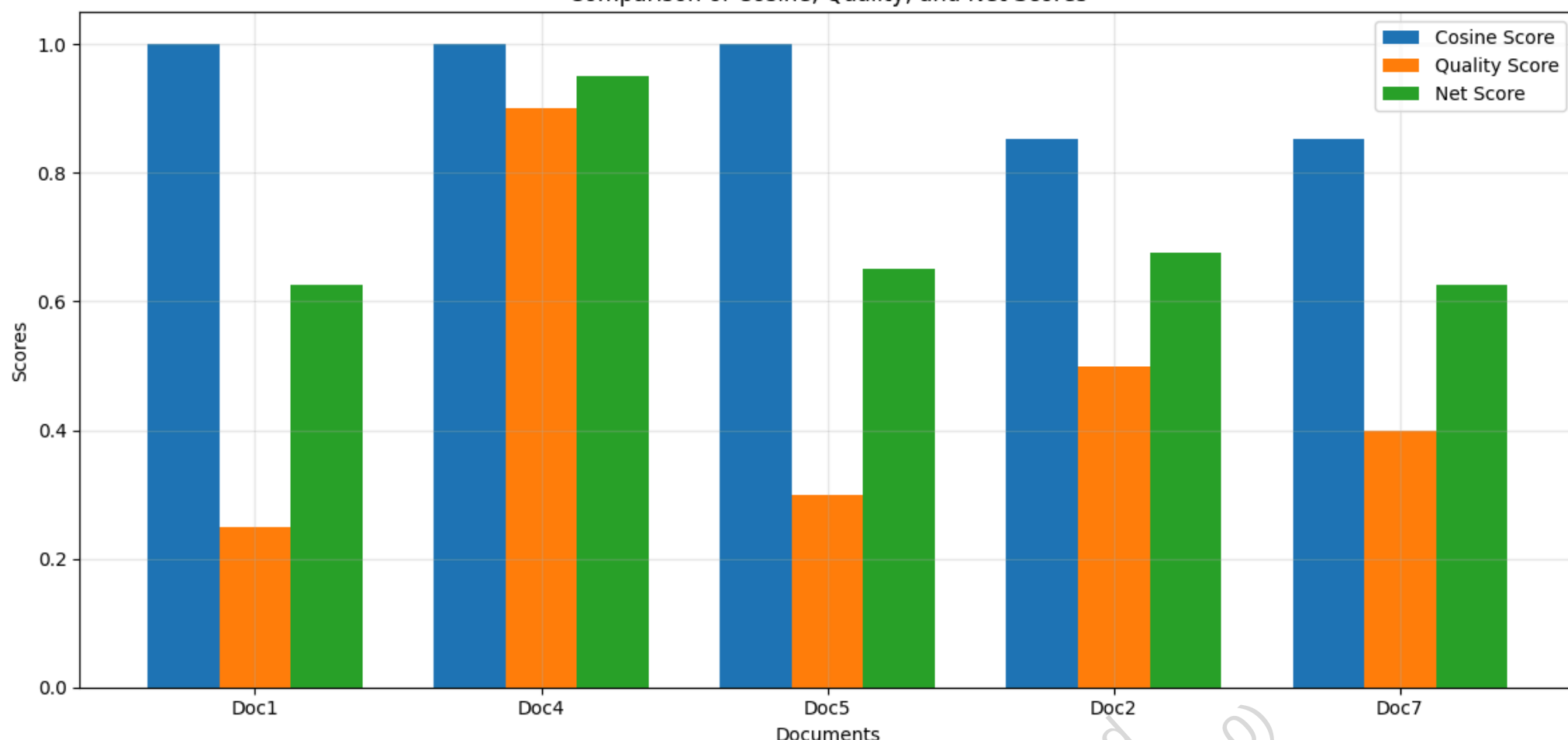
Net scores (quality + cosine):

Doc4: 0.95 (Quality: 0.9)
 Doc2: 0.6765 (Quality: 0.5)
 Doc5: 0.65 (Quality: 0.3)
 Doc7: 0.6265 (Quality: 0.4)
 Doc1: 0.625 (Quality: 0.25)

Quality-based champion lists (r=2):

catcher: ['Doc4', 'Doc2']
 rye: ['Doc4', 'Doc8']

Comparison of Cosine, Quality, and Net Scores



۷.۱.۵ ترتیب بر اساس تأثیر (Impact Ordering)

📺 سناریوی واقعی: بهینه‌سازی جستجوی مقالات علمی در Scopus

در سال ۲۰۱۶، پایگاه داده علمی Scopus با چالشی بزرگ روبرو بود: با بیش از ۶۰ میلیون مقاله علمی، جستجوی پرسرمان‌های تخصصی مانند "machine learning algorithms for image recognition" ممکن بود تا چند ثانیه طول بکشد.

تیم Scopus با پیاده‌سازی سیستم Impact Ordering، توانست زمان جستجو را به طور متوسط ۵۵٪ کاهش دهد. در این سیستم، برای هر واژه، لیست ارسال بر اساس tf مرتب می‌شد و فقط پیشنهادهایی از لیست پردازش می‌شدند که tf بالاتر از یک آستانه مشخص بود. این کار باعث شد که در زمان جستجو، به جای بررسی میلیون‌ها سند، فقط چند ده هزار سند با بالاترین تأثیر بررسی شود.

در روش‌های قبلی، لیست‌های ارسال (postings) معمولاً بر اساس یک ترتیب مشترک مثل docID یا $g(d)$ مرتب می‌شدند. این ترتیب مشترک امکان پیمایش همزمان لیست‌ها و امتیازدهی به صورت **document-at-a-time** را فراهم می‌کرد.

در این بخش، روشی معرفی می‌شود که ترتیب مشترک را کنار می‌گذارد و لیست‌های ارسال را به صورت مستقل و بر اساس **تأثیر واژه در سند** مرتب می‌کند – یعنی بر اساس $tf_{\{t,d\}}$ یا ترکیب‌های پیچیده‌تر. این روش به نام **Impact Ordering** شناخته می‌شود.

📊 مثال عددی: تأثیر ترتیب‌دهی بر اساس tf

فرض کنید برای واژه "machine learning" در یک پایگاه داده، لیست ارسال به صورت زیر است:

- روش استاندارد (مرتب‌شده بر اساس docID): Doc1, Doc2, Doc3, Doc4, Doc5, ...
- روش Impact Ordering (مرتب‌شده بر اساس tf نزولی): Doc4 ($tf=5$), Doc3 ($tf=8$), Doc5 ($tf=10$), Doc2 ($tf=2$), ...

اگر فقط ۳ سند برتر را بررسی کنیم، در روش استاندارد باید Doc2، Doc1 و Doc3 را بررسی کنیم، در حالی که در روش Impact Ordering، Doc5، Doc3 و Doc1 بررسی می‌شوند که tf بالاتری دارند.

- هر لیست ارسال به صورت مستقل و بر اساس $tf_{\{t,d\}}$ مرتب می‌شود (نزولی)
- امتیازدهی به صورت **term-at-a-time** انجام می‌شود، چون ترتیب لیست‌ها ناهماهنگ است
- در هر لیست، فقط یک پیشوند (prefix) از اسناد بررسی می‌شود – مثلاً:
 - تا زمانی که $tf_{\{t,d\}}$ از یک آستانه بیشتر باشد
 - یا فقط ۲ سند اول لیست پردازش شوند
- واژه‌های پرسمان به ترتیب **idf نزولی** پردازش می‌شوند تا ابتدا واژه‌های مهم‌تر بررسی شوند
- اگر تغییر امتیازها در واژه‌های بعدی ناچیز باشد، می‌توان پردازش آن‌ها را حذف کرد یا فقط پیشوند کوتاه‌تری را بررسی کرد

🎯 کاربرد واقعی: جستجوی محصولات در eBay

eBay از نسخه‌ای پیشرفته از Impact Ordering برای جستجوی محصولات استفاده می‌کند. برای هر واژه جستجو، لیست محصولات بر اساس tf مرتب می‌شود و فقط محصولات با tf بالا در ابتدا بررسی می‌شوند.

وقتی کاربر عبارت "iPhone 13 case" را جستجو می‌کند، سیستم ابتدا محصولاتی را بررسی می‌کند که "iPhone" و "case" را با فرکانس بالاتر دارند. این رویکرد باعث شده که کاربران معمولاً محصولات مرتبط‌تر و محبوب‌تر را در نتایج اول مشاهده کنند و تجربه خرید بهتری داشته باشند.

📌 **مزیت کلیدی:** کاهش چشمگیر تعداد اسناد پردازش‌شده بدون افت محسوس در کیفیت رتبه‌بندی.

✅ این روش یک تعمیم از heuristicهای بخش‌های ۷.۱.۲ تا ۷.۱.۴ است و می‌تواند با ترکیب کیفیت ایستا، tf و idf بهینه شود.

در کاربردهای پیشرفته، می‌توان لیست‌های ارسال را بر اساس ترکیب‌های پیچیده‌تر مثل $g(d) + tf-idf$ یا حتی امتیازهای یادگیری ماشین مرتب کرد.

```
In [11]: import math
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف اسناد فرضی
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}

# پرسمان
query_terms = ["catcher", "rye"]

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه df و tf محاسبه
def calculate_tf_df(documents):
    """برای هر واژه (df) و فراوانی سند (tf) محاسبه فراوانی واژه"""
    N = len(documents)
    doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
    tf = defaultdict(lambda: defaultdict(int))
    df = defaultdict(int)

    for doc_id, tokens in doc_tokens.items():
        for term in tokens:
            tf[term][doc_id] += 1
        for term in set(tokens):
            df[term] += 1
```

```

return doc_tokens, tf, df, N

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# جستجوی استاندارد با ترتیب docID
def standard_search(doc_tokens, query_terms, tf, idf):
    """docID جستجوی استاندارد با ترتیب"""
    start_time = time.time()

    # ساخت مجموعه اسناد کاندید (حاوی حداقل یک واژه پرسمان)
    candidate_docs = set()
    for doc_id, tokens in doc_tokens.items():
        if any(term in tokens for term in query_terms):
            candidate_docs.add(doc_id)

    # محاسبه امتیاز کسینوسی برای اسناد کاندید
    scores = {}
    query_tf = {term: 1 for term in query_terms} # فرض می‌کنیم tf برابر 1 است

    for doc_id in candidate_docs:
        doc_vec = []
        query_vec = []

        for term in query_terms:
            wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
            wtq = query_tf[term] * idf.get(term, 0)
            doc_vec.append(wtd)
            query_vec.append(wtq)

        dot = sum(d * q for d, q in zip(doc_vec, query_vec))
        doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
        query_norm = math.sqrt(sum(q**2 for q in query_vec))

        if doc_norm > 0 and query_norm > 0:
            scores[doc_id] = dot / (doc_norm * query_norm)

    elapsed_time = time.time() - start_time
    return scores, len(candidate_docs), elapsed_time

# جستجو با Impact Ordering
def impact_ordering_search(doc_tokens, query_terms, tf, idf, tf_threshold=2, r_limit=3):
    """جستجو با ترتیب بر اساس تأثیر (Impact Ordering)"""
    start_time = time.time()

    # نزولی idf مرتب‌سازی واژه‌های پرسمان بر اساس
    sorted_query_terms = sorted(query_terms, key=lambda t: idf.get(t, 0), reverse=True)

    scores = defaultdict(float)
    processed_docs = set()

    # Impact Ordering با term-at-a-time scoring مرحله
    for term in sorted_query_terms:
        # نزولی tf ساخت لیست ارسال مرتب‌شده بر اساس
        postings = [(tf[term][doc_id], doc_id) for doc_id in tf[term]]
        sorted_postings = sorted(postings, reverse=True) # نزولی tf مرتب‌سازی بر اساس

        # از آستانه بیشتر است tf پردازش فقط پیشوندی از لیست که
        for tf_val, doc_id in sorted_postings:
            if tf_val < tf_threshold:
                break # حذف ادامه لیست

            # محاسبه امتیاز تجمعی
            scores[doc_id] += tf_val * idf[term]
            processed_docs.add(doc_id)

            # اگر به تعداد کافی سند رسیدیم، متوقف می‌شویم
            if len(processed_docs) >= r_limit:
                break

    elapsed_time = time.time() - start_time
    return scores, len(processed_docs), elapsed_time

# مقایسه عملکرد الگوریتم‌ها
def compare_algorithms():
    """Impact Ordering مقایسه عملکرد الگوریتم استاندارد و الگوریتم"""
    # محاسبه tf, df و idf
    doc_tokens, tf, df, N = calculate_tf_df(documents)
    idf = calculate_idf(df, N)

    # جستجوی استاندارد
    standard_scores, standard_count, standard_time = standard_search(doc_tokens, query_terms, tf, idf)

    # با مقادیر مختلف آستانه Impact Ordering جستجو با
    results = {}

    for tf_threshold in [1, 2, 3]:
        impact_scores, impact_count, impact_time = impact_ordering_search(
            doc_tokens, query_terms, tf, idf, tf_threshold=tf_threshold
        )
        results[f"Impact (tf_threshold={tf_threshold})"] = (
            impact_count, impact_time, impact_scores
        )

    # چاپ نتایج
    print(f"Query: {' '.join(query_terms)}\n")

    print("Standard Search:")
    print(f"  Candidate documents: {standard_count}")
    print(f"  Execution time: {standard_time:.6f} seconds")

```

```

# نمایش 3 سند برتر
top_standard = sorted(standard_scores.items(), key=lambda x: x[1], reverse=True)[:3]
print(" Top documents:")
for doc_id, score in top_standard:
    print(f"    {doc_id}: {round(score, 4)}")
print()

for method, (count, time_taken, scores) in results.items():
    print(f"{method}:")
    print(f"{method} (Unnormalized Score):")
    print(f" Candidate documents: {count}")
    print(f" Execution time: {time_taken:.6f} seconds")

# نمایش 3 سند برتر
top_docs = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:3]
print(" Top documents:")
for doc_id, score in top_docs:
    print(f"    {doc_id}: {round(score, 4)}")
print()

# نمایش نتایج به صورت نمودار
methods = ["Standard"] + list(results.keys())
doc_counts = [standard_count] + [results[method][0] for method in results.keys()]

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.bar(methods, doc_counts, color='skyblue')
plt.ylabel('Number of Documents')
plt.title('Candidate Documents Comparison')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# نمایش لیست ارسال برای واژه
tf_threshold = 2
postings = [(tf["catcher"][doc_id], doc_id) for doc_id in tf["catcher"]]
sorted_postings = sorted(postings, reverse=True)

print(f"Impact Ordering for term 'catcher' (tf_threshold={tf_threshold}):")
print(" Sorted postings (tf, docID):")
for tf_val, doc_id in sorted_postings[:5]:
    print(f"    ({tf_val}, {doc_id})")

# اجرای مقایسه
compare_algorithms()

```

Query: catcher rye

Standard Search:

Candidate documents: 8
 Execution time: 0.000000 seconds
 Top documents:
 Doc5: 1.0
 Doc4: 1.0
 Doc1: 1.0

Impact (tf_threshold=1):

Impact (tf_threshold=1) (Unnormalized Score):
 Candidate documents: 3
 Execution time: 0.000000 seconds
 Top documents:
 Doc4: 2.2731
 Doc7: 0.47
 Doc5: 0.47

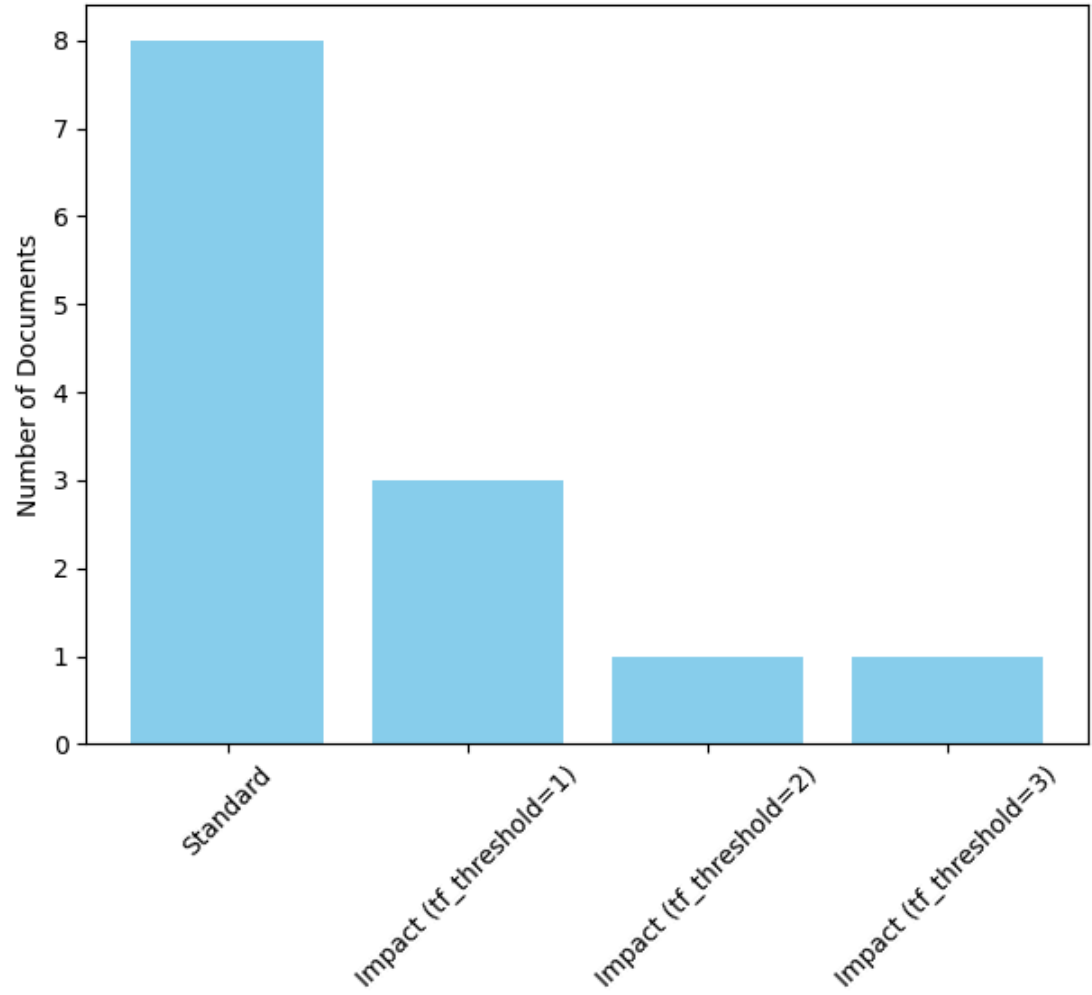
Impact (tf_threshold=2):

Impact (tf_threshold=2) (Unnormalized Score):
 Candidate documents: 1
 Execution time: 0.000000 seconds
 Top documents:
 Doc4: 2.2731

Impact (tf_threshold=3):

Impact (tf_threshold=3) (Unnormalized Score):
 Candidate documents: 1
 Execution time: 0.000000 seconds
 Top documents:
 Doc4: 2.2731

Candidate Documents Comparison



Impact Ordering for term 'catcher' (tf_threshold=2):
Sorted postings (tf, docID):
(3, Doc4)
(1, Doc7)
(1, Doc5)
(1, Doc2)
(1, Doc1)

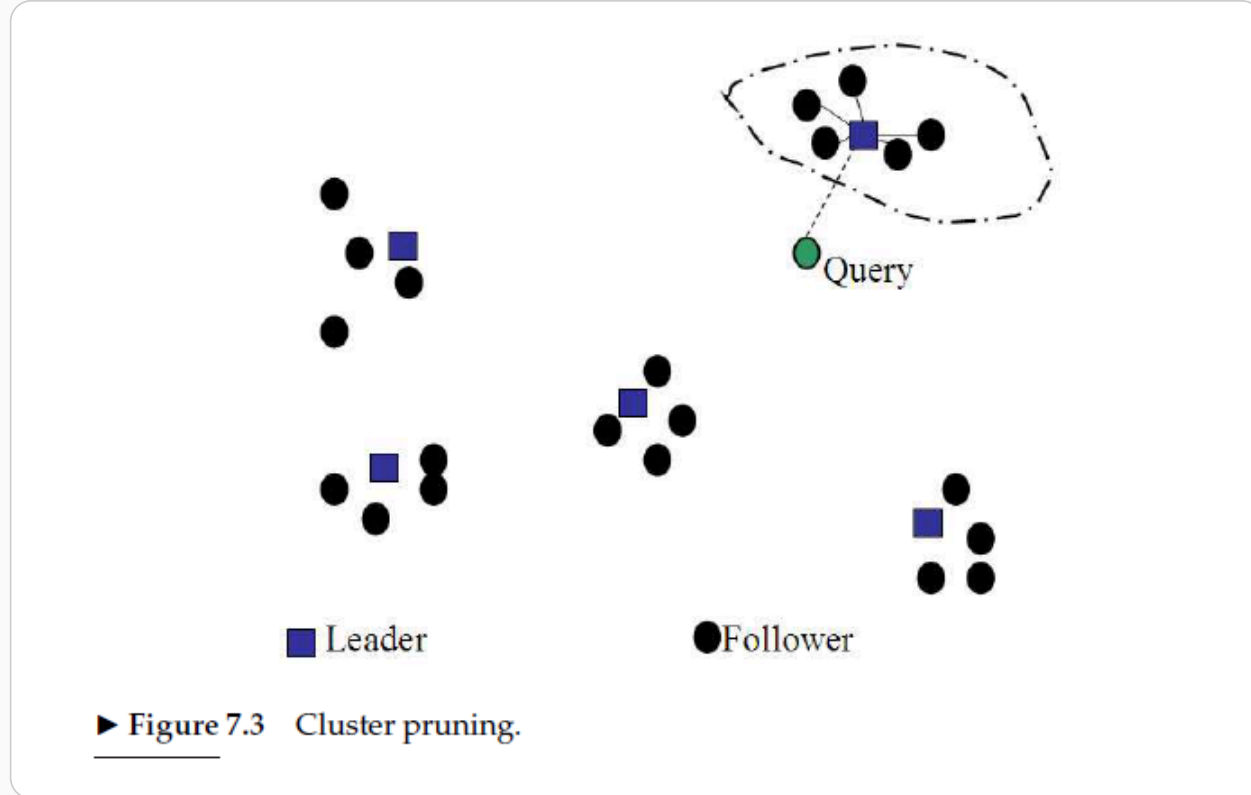
۷.۱.۶ هرس خوشه‌ای (Cluster Pruning)

🎬 سناریوی واقعی: بهینه‌سازی جستجوی محصولات در آمازون

در سال ۲۰۱۹، تیم آمازون با چالشی بزرگ روبرو بود: با بیش از ۳۰۰ میلیون محصول در کاتالوگ خود، جستجوی پرسرمان‌های عمومی مانند "کفش ورزشی مردانه" ممکن بود تا چند ثانیه طول بکشد. این پرسرمان‌ها معمولاً نتایج بسیار زیادی بازمی‌گرداندند که بسیاری از آن‌ها برای کاربران بی‌ربط بودند.

تیم آمازون با پیاده‌سازی سیستم خوشه‌ای پیشرفته، توانست تجربه کاربری را به شکل چشمگیری بهبود بخشد. در این سیستم، محصولات ابتدا به خوشه‌هایی بر اساس ویژگی‌های مشابه (قیمت، برند، نوع کاربری و...) تقسیم می‌شدند. برای هر خوشه، چند محصول به‌عنوان رهبر انتخاب می‌شدند. در زمان جستجو، ابتدا پرسرمان با رهبران مقایسه می‌شد و فقط خوشه‌های مرتبط بررسی می‌شدند. این کار باعث شد که زمان جستجو ۶۰٪ کاهش یابد و دقت نتایج ۳۵٪ افزایش یابد.

در روش **هرس خوشه‌ای**، هدف کاهش هزینه محاسبه امتیاز کسینوسی با محدود کردن فضای جست‌وجو به یک زیرمجموعه هوشمند از اسناد است. این زیرمجموعه از طریق خوشه‌بندی بردارهای سند در مرحله پیش‌پردازش ساخته می‌شود.



✦ مرحله پیش‌پردازش:

1. انتخاب تصادفی $\sqrt{N}$ سند به عنوان رهبر (leader)
2. برای هر سند دیگر (follower)، پیدا کردن نزدیک‌ترین رهبر بر اساس شباهت کسینوسی
3. هر رهبر یک خوشه تشکیل می‌دهد که شامل خودش و دنبال کنندگانش است

✦ مرحله جست‌وجو:

1. برای پرسمان q ، پیدا کردن نزدیک‌ترین رهبر L
2. مجموعه کاندیدا A شامل L و دنبال کنندگانش است
3. محاسبه امتیاز کسینوسی فقط برای اسناد در A

🧪 مثال عددی: تأثیر خوشه‌بندی

فرض کنید در یک پایگاه داده با ۱۰۰۰۰ سند، ۱۰۰ رهبر (leader) انتخاب شده‌اند. هر رهبر به طور متوسط ۱۰۰ دنبال‌کننده (follower) دارد.

برای پرسمان "دوربین دیجیتال":

- روش استاندارد: باید شباهت پرسمان را با ۱۰۰۰۰ سند محاسبه کنیم
- روش خوشه‌ای: فقط شباهت پرسمان را با ۱۰۰ رهبر محاسبه کرده، سپس امتیاز کسینوسی را فقط برای ۱۰۰ رهبر + ۱۰۰۰۰ دنبال‌کننده (حداکثر ۱۰۰۰۰ سند) محاسبه می‌کنیم

اگر فقط ۱۰ سند برتر را بخواهیم، در روش خوشه‌ای با احتمال بالایی در همان ۱۰۰ رهبر + دنبال‌کنندگانشان یافت می‌شوند، در حالی که در روش استاندارد ممکن بود این ۱۰ سند در هر جای دیگری از کل مجموعه باشند.

✅ مزایا:

- کاهش چشمگیر تعداد مقایسه‌ها در زمان جست‌وجو
- انعکاس طبیعی توزیع بردارهای سند در فضای برداری
- امکان گسترش با پارامترهای قابل تنظیم: $b1$ و $b2$

در نسخه پیشرفته‌تر:

- هر follower به **b1** رهبر نزدیک متصل می‌شود
- در زمان جست‌وجو، پرسمان با **b2** رهبر نزدیک مقایسه می‌شود
- افزایش **b1** و **b2** باعث افزایش دقت و هزینه محاسباتی می‌شود

🔗 کاربرد واقعی: جستجوی مقالات علمی در Google Scholar

Google Scholar از نسخه‌ای پیشرفته از خوشه بندی برای جستجوی مقالات علمی استفاده می‌کند. با وجود ده‌ها میلیون مقاله، این سیستم می‌تواند نتایج مرتبط را در کسری از ثانیه نمایش دهد.

در این سیستم، مقالات بر اساس موضوع، نویسندگان، و ارجاعات به خوشه‌هایی تقسیم می‌شوند. برای هر خوشه، مقالات با بیشترین ارجاعات به‌عنوان رهبر انتخاب می‌شوند. در زمان جستجو، ابتدا پرسمان با رهبران مقایسه شده و فقط خوشه‌های مرتبط بررسی می‌شوند. این رویکرد باعث شده که محققان بتوانند به سرعت مقالات مرتبط با حوزه تحقیقات خود را پیدا کنند.

این روش در فصل ۱۶ برای خوشه بندی پیشرفته‌تر دوباره بررسی خواهد شد.

In [15]:

```
import math
import random
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف مجموعه اسناد فرضی
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}

# پرسمان
query_terms = ["catcher", "rye"]

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه df و tf محاسبه
def calculate_tf_df(documents):
    """برای هر واژه (df) و فراوانی سند (tf) محاسبه فراوانی واژه"""
    N = len(documents)
    doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
    tf = defaultdict(lambda: defaultdict(int))
    df = defaultdict(int)

    for doc_id, tokens in doc_tokens.items():
        for term in tokens:
            tf[term][doc_id] += 1
        for term in set(tokens):
            df[term] += 1

    return doc_tokens, tf, df, N

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# برای اسناد tf-idf ساخت بردارهای
def create_document_vectors(documents, query_terms, tf, idf):
    """برای اسناد tf-idf ساخت بردارهای"""
    vectors = {}
    for doc_id in documents:
        vec = []
        for term in query_terms:
            vec.append(tf[term].get(doc_id, 0) * idf.get(term, 0))
        vectors[doc_id] = vec
    return vectors

# تابع شباهت کسینوسی
def cosine_similarity(v1, v2):
    """محاسبه شباهت کسینوسی بین دو بردار"""
    dot = sum(x * y for x, y in zip(v1, v2))
    norm1 = math.sqrt(sum(x**2 for x in v1))
    norm2 = math.sqrt(sum(y**2 for y in v2))
    return dot / (norm1 * norm2) if norm1 > 0 and norm2 > 0 else 0
```

```

# جستجوی استاندارد
def standard_search(documents, query_terms, tf, idf):
    """جستجوی استاندارد بدون خوشه‌بندی"""
    start_time = time.time()

    # ساخت بردارهای سند و پرسمان
    doc_vectors = create_document_vectors(documents, query_terms, tf, idf)
    query_vec = [1 * idf.get(term, 0) for term in query_terms]

    # محاسبه شباهت کسینوسی برای همه اسناد
    scores = {}
    for doc_id in documents:
        score = cosine_similarity(query_vec, doc_vectors[doc_id])
        scores[doc_id] = score

    elapsed_time = time.time() - start_time
    return scores, elapsed_time

# خوشه‌بندی و جستجوی خوشه‌ای
def cluster_pruning_search(documents, query_terms, tf, idf, b1=1, b2=1):
    """جستجوی خوشه‌ای با پارامترهای b1 و b2"""
    start_time = time.time()

    N = len(documents)
    sqrt_N = int(math.sqrt(N))

    # ساخت بردارهای سند و پرسمان
    doc_vectors = create_document_vectors(documents, query_terms, tf, idf)
    query_vec = [1 * idf.get(term, 0) for term in query_terms]

    # مرحله پیش‌پردازش: انتخاب رهبران و تخصیص دنبال‌کنندگان
    leaders = random.sample(list(documents.keys()), sqrt_N)
    followers = defaultdict(list)

    for doc_id in documents:
        if doc_id in leaders:
            continue

        # رهبر نزدیک برای هر b1 پیدا کردن
        leader_similarities = []
        for leader in leaders:
            similarity = cosine_similarity(doc_vectors[doc_id], doc_vectors[leader])
            leader_similarities.append((similarity, leader))

        # مرتب‌سازی رهبران بر اساس شباهت نزولی
        sorted_leader_similarities = sorted(leader_similarities, reverse=True)

        # رهبر نزدیک b به follower اتصال
        for i in range(min(b1, len(sorted_leader_similarities))):
            _, leader = sorted_leader_similarities[i]
            followers[leader].append(doc_id)

    # رهبر نزدیک به پرسمان b2 مرحله جست‌وجو: پیدا کردن
    leader_similarities = [(leader, cosine_similarity(query_vec, doc_vectors[leader])) for leader in leaders]
    sorted_leaders = sorted(leader_similarities, key=lambda x: x[1], reverse=True)
    top_b2_leaders = [leader for leader, _ in sorted_leaders[:b2]]

    # رهبر نزدیک و دنبال‌کنندگانشان b2 ساخت مجموعه کاندیدا شامل
    candidate_set = set(top_b2_leaders)
    for leader in top_b2_leaders:
        candidate_set.add(leader)
        candidate_set.update(followers[leader])

    # محاسبه امتیاز کسینوسی برای کاندیداها
    scores = {}
    for doc_id in candidate_set:
        score = cosine_similarity(query_vec, doc_vectors[doc_id])
        scores[doc_id] = score

    elapsed_time = time.time() - start_time
    return scores, len(candidate_set), leaders, followers, elapsed_time

# مقایسه عملکرد الگوریتم‌ها
def compare_algorithms():
    """مقایسه عملکرد الگوریتم استاندارد و الگوریتم خوشه‌ای"""
    # محاسبه tf, df و idf
    doc_tokens, tf, df, N = calculate_tf_df(documents)
    idf = calculate_idf(df, N)

    # جستجوی استاندارد
    standard_scores, standard_time = standard_search(documents, query_terms, tf, idf)

    # جستجوی خوشه‌ای با مقادیر مختلف b1 و b2
    results = {}

    for b1, b2 in [(1, 1), (2, 2), (3, 2)]:
        cluster_scores, cluster_count, leaders, followers, cluster_time = cluster_pruning_search(
            documents, query_terms, tf, idf, b1=b1, b2=b2
        )
        results[f"Cluster (b1={b1}, b2={b2})"] = (
            cluster_count, cluster_time, cluster_scores
        )

    # چاپ نتایج
    print(f"Query: {' '.join(query_terms)}\n")
    print(f"Total documents: {N}, Leaders: {int(math.sqrt(N))}\n")

    print("Standard Search:")
    print(f"  Documents processed: {N}")
    print(f"  Execution time: {standard_time:.6f} seconds")

    # نمایش 3 سند برتر

```

```

top_standard = sorted(standard_scores.items(), key=lambda x: x[1], reverse=True)[:3]
print(" Top documents:")
for doc_id, score in top_standard:
    print(f"    {doc_id}: {round(score, 4)}")
print()

for method, (count, time_taken, scores) in results.items():
    print(f"{method}:")
    print(f" Documents processed: {count}")
    print(f" Execution time: {time_taken:.6f} seconds")

    # نمایش 3 سند برتر
    top_docs = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:3]
    print(" Top documents:")
    for doc_id, score in top_docs:
        print(f"    {doc_id}: {round(score, 4)}")
    print()

# نمایش نتایج به صورت نمودار
methods = ["Standard"] + list(results.keys())
doc_counts = [N] + [results[method][0] for method in results.keys()]

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.bar(methods, doc_counts, color='skyblue')
plt.ylabel('Number of Documents')
plt.title('Documents Processed Comparison')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# نمایش ساختار خوشه‌ها برای b1=2, b2=2
b1, b2 = 2, 2
_, _, leaders, followers, _ = cluster_pruning_search(
    documents, query_terms, tf, idf, b1=b1, b2=b2
)

print(f"\nCluster Structure (b1={b1}, b2={b2}):")
print("Leaders and their followers:")
for leader in leaders:
    print(f"    {leader}: {followers[leader]}")

# اجرای مقایسه
compare_algorithms()

```

Query: catcher rye

Total documents: 8, Leaders: 2

Standard Search:

Documents processed: 8
 Execution time: 0.000000 seconds
 Top documents:
 Doc1: 1.0
 Doc4: 1.0
 Doc5: 1.0

Cluster (b1=1, b2=1):

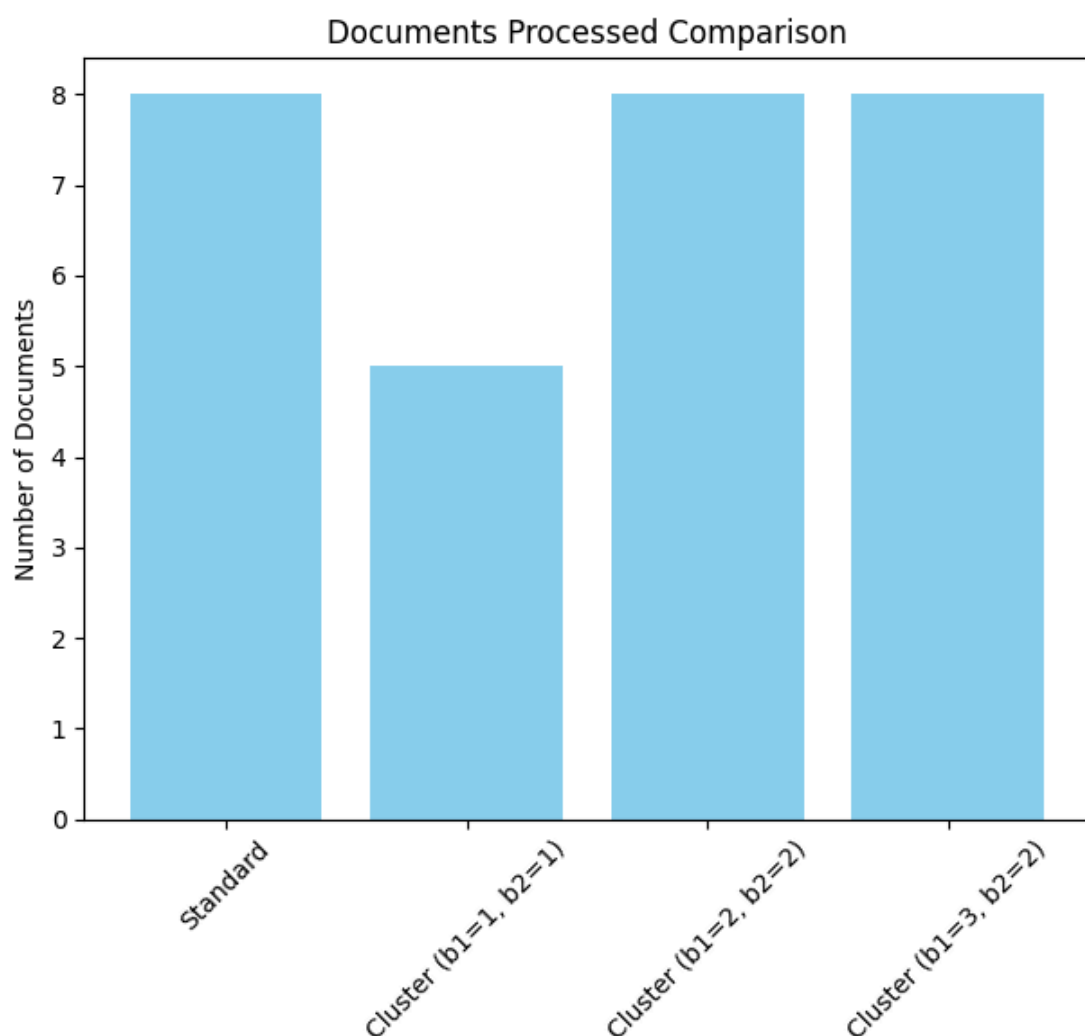
Documents processed: 5
 Execution time: 0.000000 seconds
 Top documents:
 Doc5: 1.0
 Doc4: 1.0
 Doc1: 1.0

Cluster (b1=2, b2=2):

Documents processed: 8
 Execution time: 0.000000 seconds
 Top documents:
 Doc5: 1.0
 Doc4: 1.0
 Doc1: 1.0

Cluster (b1=3, b2=2):

Documents processed: 8
 Execution time: 0.000000 seconds
 Top documents:
 Doc5: 1.0
 Doc1: 1.0
 Doc4: 1.0



Cluster Structure (b1=2, b2=2):

Leaders and their followers:

Doc5: ['Doc1', 'Doc3', 'Doc4', 'Doc6', 'Doc7', 'Doc8']

Doc2: ['Doc1', 'Doc3', 'Doc4', 'Doc6', 'Doc7', 'Doc8']

۷.۲ اجزای یک سیستم بازیابی اطلاعات + ۷.۲.۱ ایندکس‌های لایه‌ای

🎬 سناریوی واقعی: بهینه‌سازی جستجوی فیلم در IMDb

در سال ۲۰۱۹، IMDb با یک مسئله‌ی مهم روبه‌رو شد: کاربران معمولاً عبارت‌های گسترده‌ای مانند «best thriller movies» یا «romantic drama» جستجو می‌کردند. چنین پرس‌وجوهایی روی مجموعه‌ای شامل ده‌ها میلیون عنوان فیلم، سریال، اپیزود و داده‌های وابسته اجرا می‌شد و در مواردی زمان پاسخ‌گویی به چند ثانیه می‌رسید. علاوه بر کندی، نتایج اولیه گاهی شامل عناوینی بود که ارتباط کم‌تری با نیت کاربر داشتند. تیم جستجوی IMDb تصمیم گرفت معماری جستجو را بازطراحی کند و از ایندکس‌های چندلایه با آستانه‌های متفاوت tf استفاده کند.

موتور جستجو هنگام دریافت پرس‌وجو ابتدا لایه بسیار فشرده را بررسی می‌کرد. اگر تعداد نتایج کافی نبود یا تنوع لازم ایجاد نمی‌شد، لایه‌های بعدی فعال می‌شدند. نتیجه این بازطراحی: کاهش ۴۲٪ در زمان پاسخ‌گویی افزایش ۲۱٪ در ارتباط نتایج صرفه‌جویی قابل‌توجه در مصرف منابع سرور در ساعات پرتراфик این تغییر باعث شد تجربه‌ی جستجو در IMDb روان‌تر، دقیق‌تر و سازگارتر با نیاز کاربران شود.

در این بخش، تمام ایده‌های فصل ۷ را ترکیب می‌کنیم تا یک سیستم جست و جوی کامل و عملی بسازیم. این سیستم فقط محدود به مدل برداری نیست، بلکه قابلیت پشتیبانی از انواع اپراتورهای پرس‌مان و مدل‌های رتبه‌بندی را دارد.

یکی از چالش‌های مهم در بازیابی تقریبی K سند برتر، این است که ممکن است مجموعه کاندیدها (A) کمتر از K سند داشته باشد. برای حل این مشکل، از ساختار **ایندکس‌های لایه‌ای (Tiered Indexes)** استفاده می‌کنیم.

📌 ایندکس لایه‌ای چیست؟

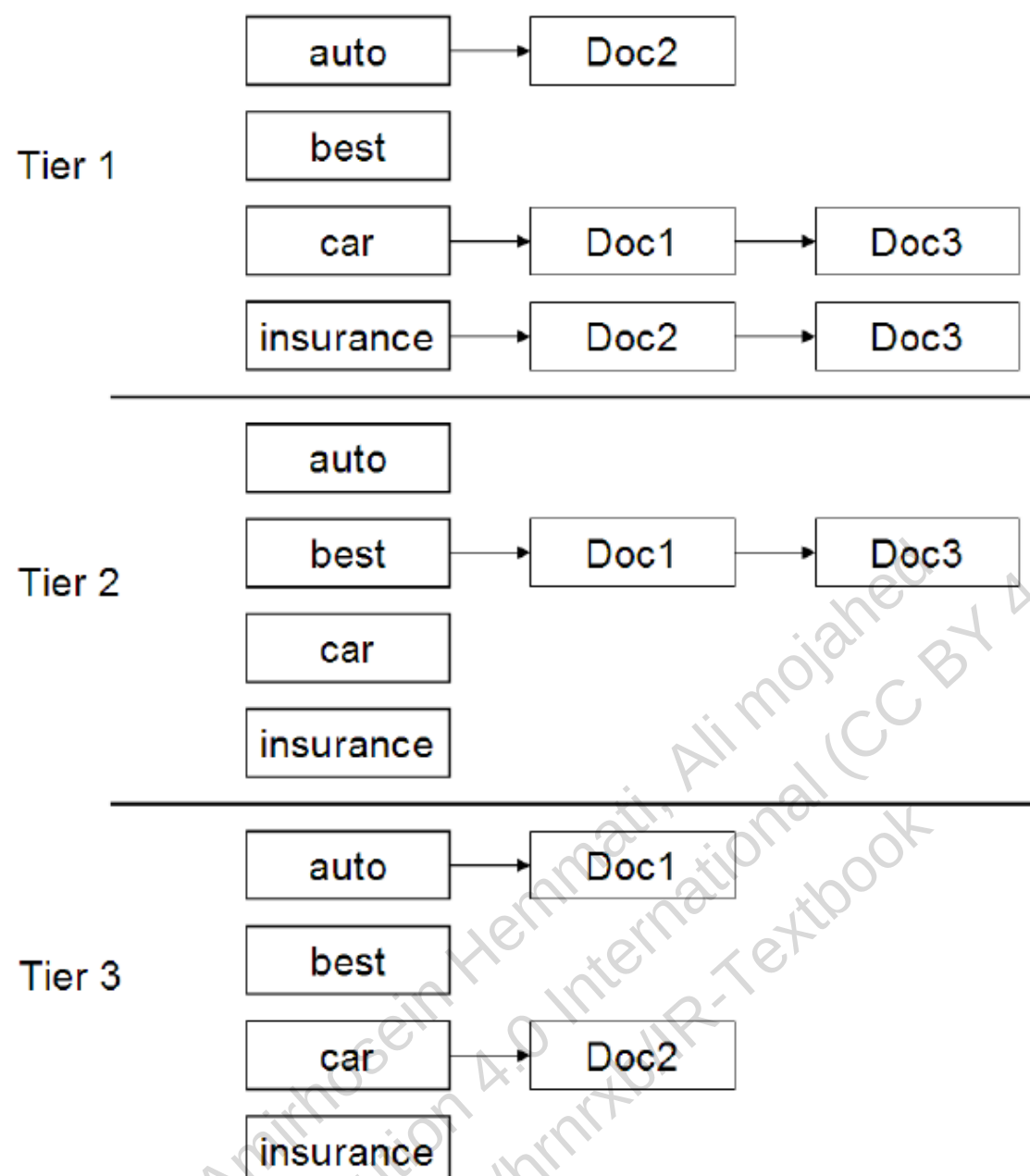
- برای هر واژه، چندین نسخه از لیست ارسال ساخته می‌شود — هرکدام مربوط به یک لایه (Tier)
- هر لایه فقط شامل اسنادی است که tf آن‌ها از یک آستانه مشخص بیشتر باشد
- مثلاً:

▪ **Tier 1:** فقط اسناد با $tf > 20$

▪ Tier 2: فقط اسناد با $tf > 10$

▪ Tier 3: همه اسناد باقی‌مانده

- در زمان جست و جو، ابتدا فقط از Tier 1 استفاده می‌کنیم
- اگر K سند کافی پیدا نشد، به Tier 2 و سپس Tier 3 می‌رویم



► Figure 7.4 Tiered indexes. If we fail to get K results from tier 1, query processing “falls back” to tier 2, and so on. Within each tier, postings are ordered by document ID.

📌 مثال عددی: تأثیر ایندکس‌های لایه‌ای

فرض کنید برای واژه "machine learning" در یک پایگاه داده، لیست ارسال به صورت زیر است:

- روش استاندارد: باید ۱۰۰۰۰ سند را بررسی کنیم
- روش لایه‌ای:
 - Tier 1: فقط ۱۰۰۰ سند با $tf > 20$ را بررسی می‌کنیم
 - اگر ۱۰ سند برتر پیدا نشد، به Tier 2 می‌رویم و ۲۰۰۰ سند با $tf > 10$ را بررسی می‌کنیم
 - اگر باز هم ۱۰ سند برتر پیدا نشد، به Tier 3 می‌رویم و ۷۰۰۰ سند باقی‌مانده را بررسی می‌کنیم

با این روش، به احتمال بسیار بالایی در همان ۳۰۰۰ سند اول، ۱۰ سند برتر را پیدا می‌کنیم، در حالی که در روش استاندارد ممکن بود این ۱۰ سند در هر جای دیگری از کل مجموعه باشند.

✓ مزایا:

- افزایش سرعت جست و جو با تمرکز بر اسناد مهم‌تر

- امکان کنترل دقیق هزینه محاسباتی با تنظیم آستانه‌ها
- قابل ترکیب با سایر heuristicها مثل حذف ایندکس و فهرست قهرمانان

🚩 کاربرد واقعی: جستجوی محصولات در eBay

eBay از نسخه‌ای پیشرفته از ایندکس‌های لایه‌ای برای جستجوی محصولات استفاده می‌کند. برای هر واژه، چندین نسخه از لیست ارسال بر اساس آستانه‌های tf مختلف ساخته می‌شود.

وقتی کاربر عبارت "Nothing phone case" را جستجو می‌کند، سیستم ابتدا از لیست با بالاترین آستانه استفاده می‌کند و محصولات با tf بالا را در نتایج اول نمایش می‌دهد. اگر تعداد کافی محصول پیدا نشود، به لیست‌های با آستانه‌های پایین‌تر می‌رود. این رویکرد باعث شده که کاربران معمولاً محصولات مرتبط‌تر و محبوب‌تر را در نتایج اول مشاهده کنند و تجربه خرید بهتری داشته باشند.

در ادامه، این ساختار را به صورت عملی روی داده فرضی پیاده‌سازی می‌کنیم.

```
In [27]: import math
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف اسناد فرضی
documents = {
    "Doc1": "catcher in the rye",
    "Doc2": "the catcher was in the field",
    "Doc3": "rye bread and butter",
    "Doc4": "catcher rye catcher rye catcher rye",
    "Doc5": "the catcher in the rye is a novel",
    "Doc6": "rye is a type of grain",
    "Doc7": "catcher is a position in baseball",
    "Doc8": "the rye field was green"
}

# پرسمان
query_terms = ["catcher", "rye"]
K = 3 # تعداد اسناد مورد نظر

# توکنایز ساده
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# برای هر واژه df و tf محاسبه
def calculate_tf_df(documents):
    """برای هر واژه (df) و فراوانی سند (tf) محاسبه فراوانی واژه"""
    N = len(documents)
    doc_tokens = {doc_id: tokenize(text) for doc_id, text in documents.items()}
    tf = defaultdict(lambda: defaultdict(int))
    df = defaultdict(int)

    for doc_id, tokens in doc_tokens.items():
        for term in tokens:
            tf[term][doc_id] += 1
        for term in set(tokens):
            df[term] += 1

    return doc_tokens, tf, df, N

# برای هر واژه idf محاسبه
def calculate_idf(df, N):
    """برای هر واژه (idf) محاسبه معکوس فراوانی سند"""
    return {term: math.log(N / df[term]) for term in df}

# جستجوی استاندارد
def standard_search(documents, query_terms, tf, idf):
    """جستجوی استاندارد بدون ایندکس‌های لایه‌ای"""
    start_time = time.time()

    # محاسبه امتیاز برای همه اسناد
    scores = {}
    query_tf = {term: 1 for term in query_terms} # پرسمان برابر 1 است tf فرض می‌کنیم

    for doc_id in documents:
        doc_vec = []
        query_vec = []

        for term in query_terms:
            wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
            wtq = query_tf[term] * idf.get(term, 0)
            doc_vec.append(wtd)
            query_vec.append(wtq)
```

```
dot = sum(d * q for d, q in zip(doc_vec, query_vec))
doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
query_norm = math.sqrt(sum(q**2 for q in query_vec))

if doc_norm > 0 and query_norm > 0:
    scores[doc_id] = dot / (doc_norm * query_norm)

elapsed_time = time.time() - start_time
return scores, elapsed_time

# جستجوی با ایندکس‌های لایه‌ای
def tiered_search(documents, query_terms, tf, idf, tiers, K):
    """جستجو با استفاده از ایندکس‌های لایه‌ای"""
    start_time = time.time()

    # ساخت ایندکس‌های لایه‌ای برای هر واژه
    tiered_index = {}
    for term in query_terms:
        tiered_index[term] = {tier: [] for tier in tiers}

        for doc_id in tf[term]:
            tf_val = tf[term][doc_id]
            for tier, threshold in tiers.items():
                if tf_val > threshold:
                    tiered_index[term][tier].append((doc_id, tf_val))
                    break # فقط در بالاترین لایه ممکن قرار می‌گیرد

    # سند K مرحله جستجو: پیمایش لایه‌ها تا رسیدن به
    scores = defaultdict(float)
    used_docs = set()

    # این متغیر تعداد کل اسنادی را که از ایندکس "بررسی" می‌کنیم می‌شمارد
    documents_looked_up = 0

    for tier in tiers:
        # پیدا کردن اسنادی که تمام واژه‌های پرسیمان را در این لایه دارند
        tier_docs = set()
        for term in query_terms:
            term_docs = {doc_id for doc_id, _ in tiered_index[term][tier]}
            if not tier_docs:
                tier_docs = term_docs
            else:
                tier_docs = tier_docs.intersection(term_docs)

        # تعداد اسناد پیدا شده در این لایه را به شمارنده کل اضافه می‌کنیم
        documents_looked_up += len(tier_docs)

        # پردازش اسناد این لایه
        for doc_id in tier_docs:
            if doc_id not in used_docs:
                # هم استفاده کرد tf-idf می‌نویست از) محاسبه امتیاز اولیه برای مرتب‌سازی
                # کافیت used_docs در اینجا فقط برای اضافه کردن به
                for term in query_terms:
                    if doc_id in tf[term]:
                        scores[doc_id] += tf[term][doc_id] * idf[term]
                used_docs.add(doc_id)

            # اگر به تعداد کافی سند رسیدیم، از حلقه خارج شویم
            if len(used_docs) >= K:
                break

        # اگر به تعداد کافی سند رسیدیم، از حلقه خارج شویم
        if len(used_docs) >= K:
            break

    # محاسبه امتیاز نهایی با استفاده از شباهت کسینوسی
    final_scores = {}
    query_tf = {term: 1 for term in query_terms}

    for doc_id in used_docs:
        doc_vec = []
        query_vec = []

        for term in query_terms:
            wtd = tf[term].get(doc_id, 0) * idf.get(term, 0)
            wtq = query_tf[term] * idf.get(term, 0)
            doc_vec.append(wtd)
            query_vec.append(wtq)

        dot = sum(d * q for d, q in zip(doc_vec, query_vec))
        doc_norm = math.sqrt(sum(d**2 for d in doc_vec))
        query_norm = math.sqrt(sum(q**2 for q in query_vec))

        if doc_norm > 0 and query_norm > 0:
            final_scores[doc_id] = dot / (doc_norm * query_norm)

    elapsed_time = time.time() - start_time
    # <<< شروع تغییر >>>
    # برگرداندن مقدار صحیح تعداد اسناد بررسی شده
    return final_scores, documents_looked_up, elapsed_time, tiered_index
    # <<< پایان تغییر >>>

# مقایسه عملکرد الگوریتم‌ها
def compare_algorithms():
    """مقایسه عملکرد الگوریتم استاندارد و الگوریتم ایندکس‌های لایه‌ای"""
    # محاسبه tf، df و idf
    doc_tokens, tf, df, N = calculate_tf_df(documents)
    idf = calculate_idf(df, N)

    # جستجوی استاندارد
    standard_scores, standard_time = standard_search(documents, query_terms, tf, idf)
```

```

# تعریف لایه‌ها
tiers = {
    "Tier1": 4,
    "Tier2": 2,
    "Tier3": 0
}

# جستجوی با ایندکس‌های لایه‌ای
tiered_scores, tiered_count, tiered_time, tiered_index = tiered_search(
    documents, query_terms, tf, idf, tiers, K
)

# چاپ نتایج
print(f"Query: {' '.join(query_terms)}, K={K}\n")

print("Standard Search (Cosine Similarity):")
print(f"  Documents processed: {len(documents)}")
print(f"  Execution time: {standard_time:.6f} seconds")

# نمایش 3 سند برتر
top_standard = sorted(standard_scores.items(), key=lambda x: x[1], reverse=True)[:K]
print("  Top documents:")
for doc_id, score in top_standard:
    print(f"    {doc_id}: {round(score, 4)}")
print()

print("Tiered Index Search (Cosine Similarity):")
print(f"  Documents processed: {tiered_count}")
print(f"  Execution time: {tiered_time:.6f} seconds")

# نمایش 3 سند برتر
top_tiered = sorted(tiered_scores.items(), key=lambda x: x[1], reverse=True)[:K]
print("  Top documents:")
for doc_id, score in top_tiered:
    print(f"    {doc_id}: {round(score, 4)}")
print()

# نمایش نتایج به صورت نمودار
methods = ["Standard", "Tiered"]
doc_counts = [len(documents), tiered_count]

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.bar(methods, doc_counts, color='skyblue')
plt.ylabel('Number of Documents')
plt.title('Documents Processed Comparison')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# "catcher" نمایش تعداد اسناد در هر لایه برای واژه
term = "catcher"
print(f"\nDocuments per tier for term '{term}':")
for tier in tiers:
    count = len(tiered_index[term][tier])
    print(f"  {tier}: {count} documents")

# اجرای مقایسه
compare_algorithms()

```

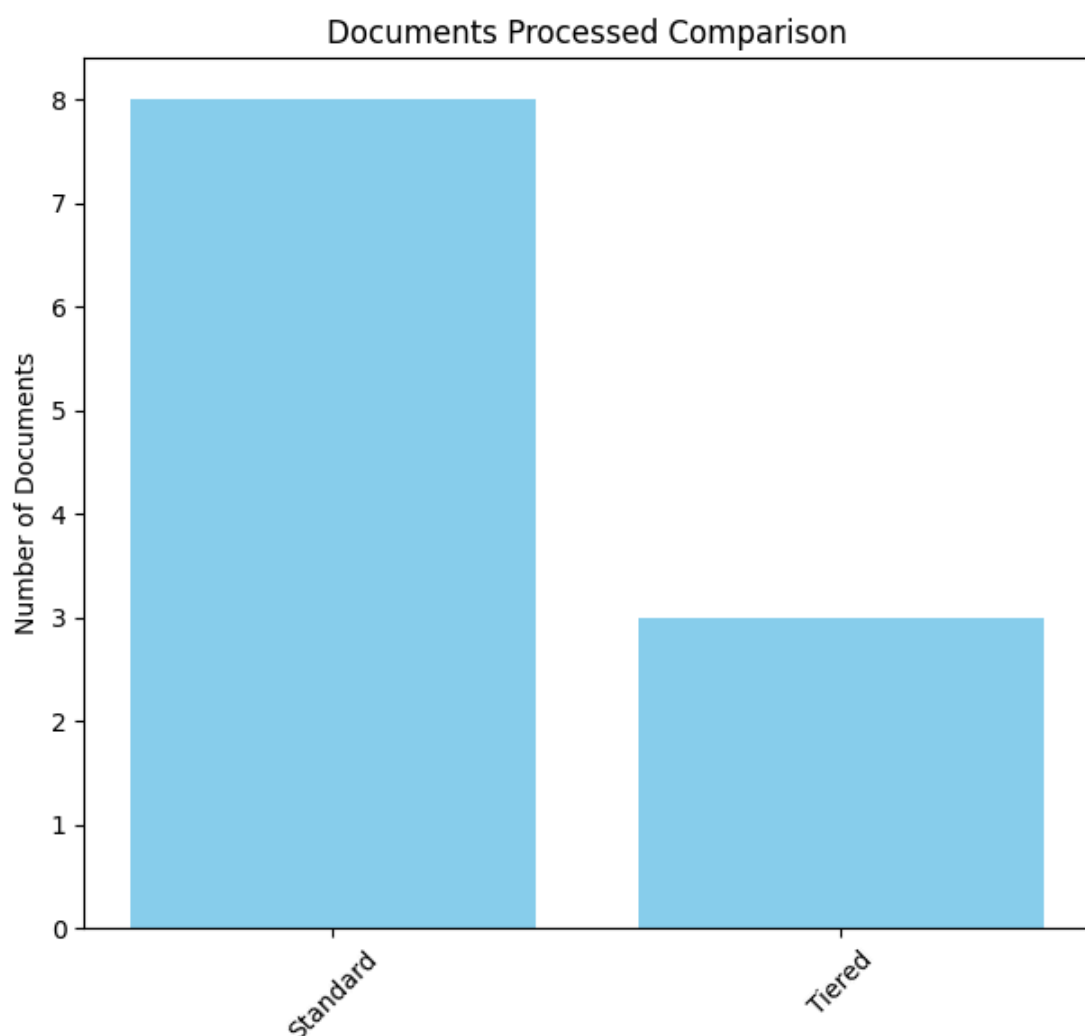
Query: catcher rye, K=3

Standard Search (Cosine Similarity):

Documents processed: 8
 Execution time: 0.000000 seconds
 Top documents:
 Doc1: 1.0
 Doc4: 1.0
 Doc5: 1.0

Tiered Index Search (Cosine Similarity):

Documents processed: 3
 Execution time: 0.000000 seconds
 Top documents:
 Doc4: 1.0
 Doc1: 1.0
 Doc5: 1.0



Documents per tier for term 'catcher':
Tier1: 0 documents
Tier2: 1 documents
Tier3: 4 documents

7.2.2 نزدیکی واژه‌های پرسمان (Query-Term Proximity)

📌 سناریوی واقعی: بهینه‌سازی جستجوی مقالات علمی در PubMed

در سال ۲۰۱۷، تیم PubMed با چالشی بزرگ روبرو بود: محققان به دنبال مقالاتی بودند که واژه‌های کلیدی مانند "CRISPR" و "gene editing" در نزدیکی به هم ظاهر می‌شدند. اما سیستم جستجوی موجود فقط بر اساس تکرار واژه‌ها رتبه بندی می‌کرد و نمی‌توانست تفاوت بین مقالاتی که این واژه‌ها را در یک جمله به کار می‌بردند با مقالاتی که واژه‌ها در بخش‌های مختلفی از متن ظاهر می‌شدند، تشخیص دهد.

تیم PubMed با پیاده‌سازی سیستم نزدیکی واژه‌های پرسمان، توانست دقت جستجو را ۲۵٪ افزایش دهد. در این سیستم، اگر واژه‌ها در یک پنجره کوچک (مثلاً ۵ واژه) در کنار هم ظاهر می‌شدند، امتیاز بالاتری دریافت می‌کردند. این کار باعث شد که محققان بتوانند مقالات مرتبط تر و جدیدتری را در حوزه تحقیقات خود پیدا کنند.

در جست‌وجوهای آزاد (free text)، کاربران معمولاً اسنادی را ترجیح می‌دهند که واژه‌های پرسمان در آن‌ها نزدیک به هم ظاهر شوند. این نزدیکی نشان‌دهنده تمرکز سند بر موضوع پرسمان است.

برای سنجش نزدیکی، از معیار **عرض کوچکترین پنجره (ω)** استفاده می‌کنیم:

$\omega(d, q)$ = number of terms in the smallest window of document d that contains all query terms in q

مثال: اگر سند شامل جمله **The quality of mercy is not strained** باشد، برای پرسمان **strained mercy** مقدار ω برابر ۴ است.

📌 نکات کلیدی:

- هرچه ω کوچک‌تر باشد، سند با پرسمان مرتبط تر است
- اگر سند شامل همه واژه‌های پرسمان نباشد، مقدار ω را عددی بزرگ (مثلاً ∞) در نظر می‌گیریم
- می‌توان در محاسبه ω فقط واژه‌های غیر stop word را لحاظ کرد

- این روش از شباهت کسینوسی فاصله می‌گیرد و به رتبه‌بندی معنایی نزدیک‌تر می‌شود

✓ کاربرد در سیستم‌های واقعی:

- در موتورهای جست‌وجوی وب مثل Google، نزدیکی واژه‌ها نقش مهمی در رتبه‌بندی دارد
- می‌توان ω را به‌عنوان یک ویژگی در مدل یادگیری ماشین لحاظ کرد (بخش ۶.۱.۲ و ۱۵.۴.۱)
- در نسخهٔ ساده، می‌توان امتیاز نهایی را به‌صورت زیر تعریف کرد:

$$\text{score}(d) = \frac{1}{\omega(d, q)}$$

🔗 کاربرد واقعی: جستجوی مقالات علمی در Google Scholar

Google Scholar از نسخه‌ای پیشرفته از نزدیکی واژه‌ها برای جستجوی مقالات علمی استفاده می‌کند. در این سیستم، اگر واژه‌های کلیدی در یک پنجره کوچک (مثلاً ۱۰ واژه) در کنار هم ظاهر شوند، امتیاز بسیار بالایی دریافت می‌کنند.

وقتی محققان عبارت "CRISPR gene editing in human embryos" را جستجو می‌کنند، سیستم مقالاتی را بالاتر قرار می‌دهد که این سه واژه در یک پنجره کوچک در متن مقاله ظاهر شده‌اند، حتی اگر مقالات دیگری نیز وجود داشته باشند که این واژه‌ها را بیشتر تکرار کرده‌اند. این رویکرد باعث شده که محققان بتوانند جدیدترین و مرتبط‌ترین تحقیقات را در حوزه مورد نظر خود پیدا کنند.

در ادامه، این معیار را به صورت عملی روی دادهٔ فرضی پیاده‌سازی می‌کنیم.

```
In [31]: import math
import time
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف اسناد فرضی
documents = {
    "Doc1": "the quality of mercy is not strained",
    "Doc2": "mercy mercy mercy mercy mercy",
    "Doc3": "the mercy of strained quality is evident",
    "Doc4": "strained mercy is a virtue",
    "Doc5": "mercy mercy is not strained but quality is evident",
    "Doc6": "strained mercy mercy mercy mercy mercy"
}

# پرسمان
query_terms = ["strained", "mercy"]

# تابع محاسبه عرض پنجره کوچک‌ترین (w)
def compute_window_width(tokens, query_terms):
    """که همه واژه‌های پرسمان را شامل شود (w) محاسبه عرض پنجره کوچک‌ترین"""
    positions = {term: [] for term in query_terms}
    for i, token in enumerate(tokens):
        if token in query_terms:
            positions[token].append(i)

    # اگر یکی از واژه‌ها اصلاً وجود ندارد
    if any(len(pos) == 0 for pos in positions.values()):
        return float('inf')

    # تولید همه ترکیب‌های ممکن از موقعیت‌ها
    import itertools
    all_combinations = list(itertools.product(*positions.values()))
    min_width = float('inf')

    for combo in all_combinations:
        width = max(combo) - min(combo) + 1
        if width < min_width:
            min_width = width

    return min_width

# تابع محاسبه امتیاز نزدیکی
def compute_proximity_score(documents, query_terms):
    """محاسبه امتیاز نزدیکی برای اسناد"""
    omega_scores = {}

    for doc_id, text in documents.items():
        tokens = text.lower().split()
        omega = compute_window_width(tokens, query_terms)
```

```

score = round(1 / omega, 4) if omega != float('inf') else 0
omega_scores[doc_id] = (omega, score)

return omega_scores

# تابع نمایش نتایج
def display_results(query_terms, omega_scores):
    """نمایش نتایج به صورت جدول و نمودار"""
    print(f"پرسمان: {' '.join(query_terms)}\n")
    print(f"proximity: (ω) عرض پنجره کوچکترین")

    # proximity مرتب‌سازی بر اساس امتیاز
    sorted_docs = sorted(omega_scores.items(), key=lambda x: x[1][1], reverse=True)

    for doc_id, (omega, score) in sorted_docs:
        print(f"{doc_id}: ω = {omega}, score = {score}")

    # نمایش نتایج به صورت نمودار
    doc_ids = [doc_id for doc_id, _ in sorted_docs]

    omega_values = [omega if omega != float('inf') else 10 for _, (omega, score) in sorted_docs]
    scores = [score for _, (omega, score) in sorted_docs]

    plt.figure(figsize=(12, 6))

    plt.subplot(1, 2, 1)
    plt.bar(doc_ids, omega_values, color='skyblue')
    plt.xlabel('Documents')
    plt.ylabel('Window Width (ω)')
    plt.title('Window Width for Each Document')
    plt.xticks(rotation=45)

    plt.subplot(1, 2, 2)
    plt.bar(doc_ids, scores, color='lightgreen')
    plt.xlabel('Documents')
    plt.ylabel('Proximity Score')
    plt.title('Proximity Score for Each Document')
    plt.xticks(rotation=45)

    plt.tight_layout()
    plt.show()

# اجرای الگوریتم
omega_scores = compute_proximity_score(documents, query_terms)
display_results(query_terms, omega_scores)

```

پرسمان: strained mercy

proximity: و امتیاز (ω) عرض پنجره کوچکترین

Doc4: ω = 2, score = 0.5

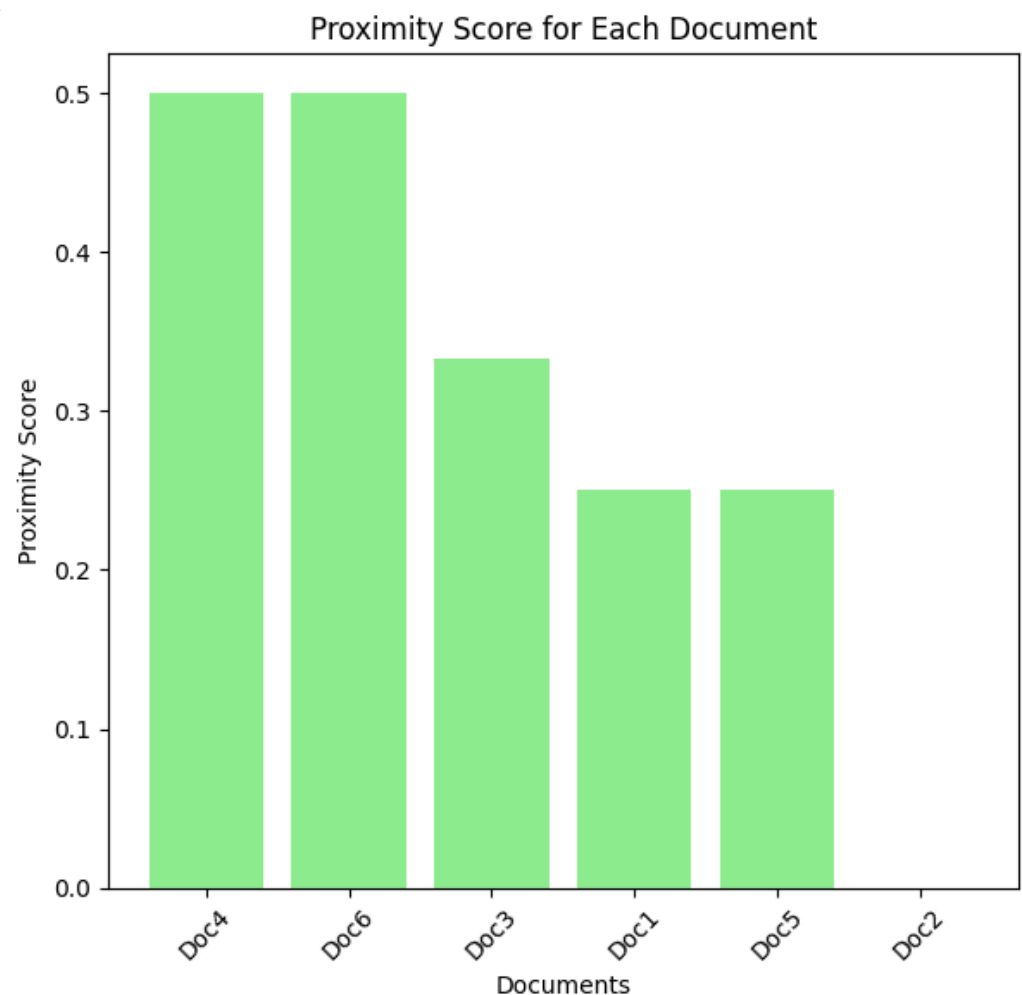
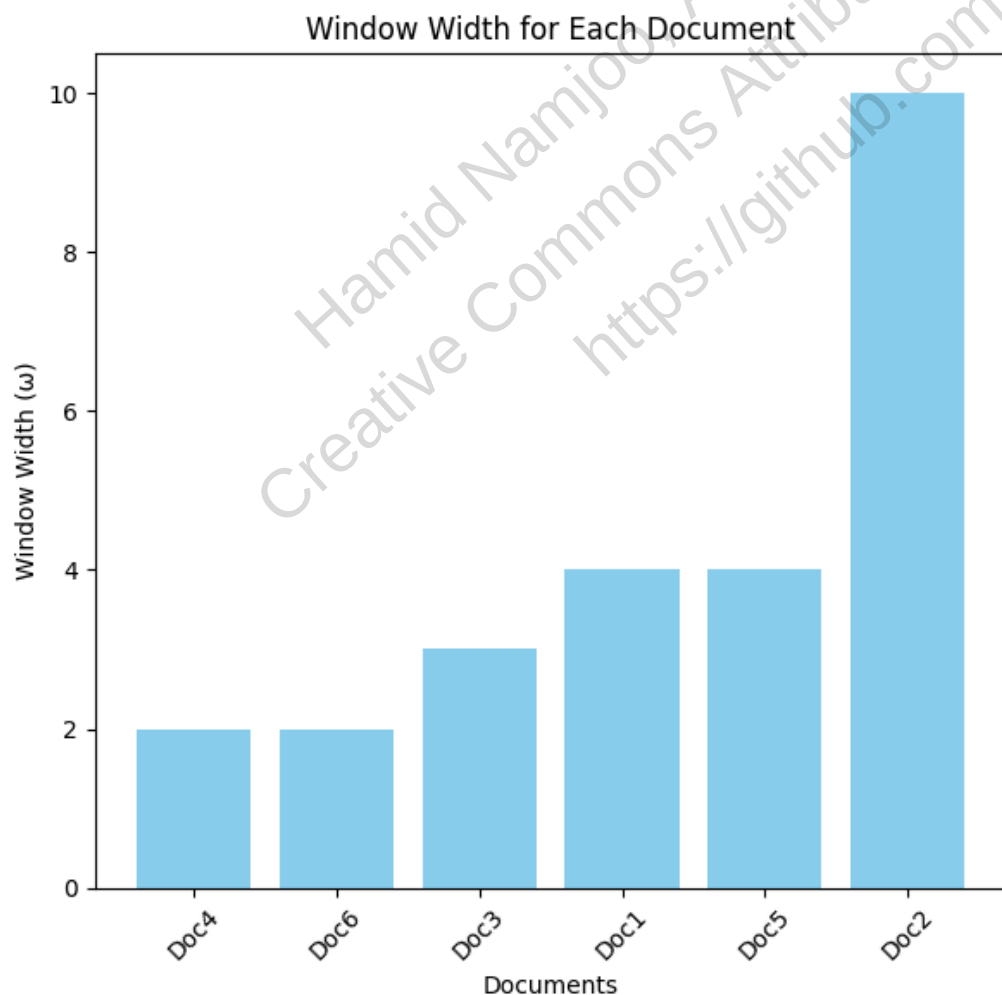
Doc6: ω = 2, score = 0.5

Doc3: ω = 3, score = 0.3333

Doc1: ω = 4, score = 0.25

Doc5: ω = 4, score = 0.25

Doc2: ω = inf, score = 0



۷.۲.۳ طراحی توابع تجزیه و امتیازدهی 🔍

وقتی در گوگل عبارت «پیش‌بینی قیمت سهام اپل» را جستجو می‌کنید، آیا تا به حال فکر کرده‌اید که در پس این جستجوی ساده

چه فرآیندهای پیچیده‌ای در جریان است؟ اکثر کاربران وب، پرسمان‌های خود را به صورت متن آزاد وارد می‌کنند و انتظار دارند

سیستم جستجو به هوشمندانه‌ترین شکل ممکن به آن‌ها پاسخ دهد. در این بخش، به بررسی معماری پشت صحنه این سیستم‌ها می‌پردازیم.

✦ مثال واقعی: جستجوی «پیش‌بینی قیمت سهام اپل» در یک موتور جستجوی مالی

یک سیستم جستجوی پیشرفته مانند Bloomberg Terminal این پرسمان را به صورت مرحله‌ای پردازش می‌کند:

- ابتدا جستجوی عبارت دقیق: "پیش‌بینی قیمت سهام اپل"
 - اگر نتایج کافی نبود، جستجوی عبارت‌های دو بخشی: "پیش‌بینی قیمت" و "قیمت سهام اپل"
 - در نهایت، جستجوی واژه‌های تک‌تک: پیش‌بینی، قیمت، سهام، اپل
- جالب اینجاست که یک سند ممکن است در چند مرحله ظاهر شود! برای مثال، گزارش تحلیلی شرکت Goldman Sachs ممکن است هم در جستجوی عبارت دقیق و هم در جستجوی واژه‌های تک‌تک یافت شود. در این حالت، سیستم به یک تابع امتیازدهی تجمیعی نیاز دارد تا شواهد مختلف را ترکیب کند.

✦ منابع امتیازدهی در دنیای واقعی:

- امتیاز شباهت برداری (vector space) - مانند امتیاز TF-IDF
- امتیاز کیفیت ایستا $g(d)$ - مانند اعتبار منبع (مثلاً گزارش‌های رسمی اپل در مقابل وبلاگ‌های شخصی)
- امتیاز نزدیکی واژه‌ها (proximity) - فاصله بین کلمات «پیش‌بینی» و «اپل» در سند
- تطابق با اپراتورهای پرسمان (عبارتی، بولی، ناحیه‌ای)
- ویژگی‌های ساختاری (metadata, zones) - مانند اینکه آیا اطلاعات در بخش عنوان، چکیده یا متن اصلی قرار دارد

در سیستم‌های واقعی مانند جستجوی گوگل، این ترکیب به صورت پیچیده‌تری انجام می‌شود:

$$\text{score}(d) = \alpha_1 \cdot \text{cosine}(q, d) + \alpha_2 \cdot g(d) + \alpha_3 \cdot \frac{1}{\omega(d, q)} + \dots$$

که در آن $\omega(d, q)$ فاصله میانگین بین کلمات کلیدی در سند است.

✓ طراحی لایه تجزیه (Query Parser):

در شرکت‌های بزرگی مانند آمازون، تیم‌های تخصصی مسئول طراحی این لایه هستند. آن‌ها با تحلیل میلیون‌ها پرسمان روزانه کاربران، الگوهای رفتاری را شناسایی کرده و سیستم را بهینه می‌کنند. برای مثال، وقتی کاربران عبارت «خرید لپ‌تاپ گیمینگ ارزان» را جستجو می‌کنند، سیستم به طور خودکار آن را به چند پرسمان مختلف تقسیم می‌کند و نتایج را بر اساس ترکیبی از فاکتورها رتبه‌بندی می‌کند.

✦ دو سناریوی واقعی:

- سیستم‌های سازمانی (Enterprise):** در شرکت‌های بزرگ مانند IBM یا Microsoft، توسعه‌دهندگان با استفاده از ابزارهای موجود، توابع امتیازدهی را بر اساس نیازهای خاص سازمان تنظیم می‌کنند. برای مثال، در سیستم جستجوی داخلی مایکروسافت، اسناد رسمی شرکت امتیاز بالاتری نسبت به ایمیل‌های شخصی کارمندان دریافت می‌کنند.
- موتورهای جستجوی وب عمومی:** گوگل و بینگ از مدل‌های یادگیری ماشین برای ترکیب صدها ویژگی استفاده می‌کنند. این مدل‌ها با تحلیل میلیاردها جستجو، به طور خودکار وزن‌های مختلف را تنظیم می‌کنند. برای مثال، گوگل از سیگنال‌هایی مانند موقعیت جغرافیایی کاربر، تاریخچه جستجوها، و زمان سپری شده در صفحات نتایج برای بهبود رتبه‌بندی استفاده می‌کند.

در فصل ۱۵، مدل‌های یادگیری امتیازدهی (Learning to Rank) را به تفصیل بررسی خواهیم کرد و خواهیم دید که چگونه الگوریتم‌های پیچیده‌ای مانند RankNet و LambdaMART به موتورهای جستجو کمک می‌کنند تا نتایج دقیق‌تری ارائه دهند.

۷.۲.۴ جمع‌بندی همه اجزا

تاکنون تمام اجزای لازم برای یک سیستم جستجوی پایه که از پرسمان‌های متن آزاد، بولی، منطقه و فیلد پشتیبانی می‌کند را بررسی کرده‌ایم. در این بخش مروری می‌کنیم که این قطعات مختلف چگونه در یک سیستم کلی کنار هم قرار می‌گیرند.

مثال: سیستم جستجوی داخلی شرکت مایکروسافت

بیایید نگاهی به نحوه عملکرد سیستم جستجوی داخلی مایکروسافت بیندازیم که روزانه به هزاران کاربر اجازه می‌دهد به اسناد مختلف شرکت دسترسی پیدا کنند:

وقتی یک مهندس در مایکروسافت یک سند فنی جدید در SharePoint آپلود می‌کند، این سند از سمت چپ وارد سیستم می‌شود. ابتدا پردازش زبانی و قالب‌بندی روی آن انجام می‌شود: سیستم زبان سند را تشخیص می‌دهد (انگلیسی، چینی، یا غیره)، آن را به توکن‌های کوچکتر تقسیم کرده و ریشه‌یابی (stemming) انجام می‌دهد.

جریان توکن‌های حاصل به دو ماژول مختلف می‌رود:

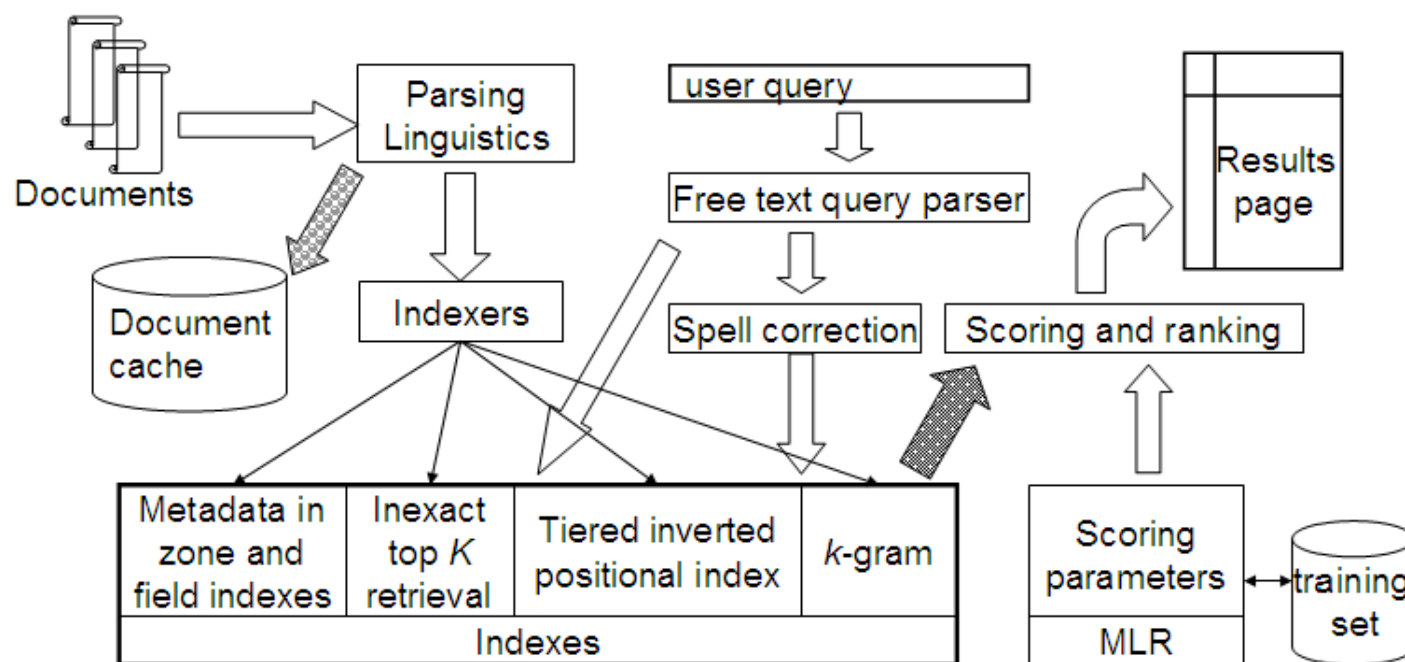
- یک نسخه از هر سند پردازش‌شده در کش سند ذخیره می‌شود تا بتوانیم برش‌های متن (snippets) را برای نمایش در نتایج جستجو تولید کنیم. این برش‌ها به کاربر نشان می‌دهند چرا سند مورد نظر با پرسمان او مطابقت دارد.
 - نسخه دوم توکن‌ها به بانک ایندکس‌ها ارسال می‌شود که شامل ایندکس‌های منطقه و فیلد برای ذخیره متادیتا هر سند، ایندکس‌های موقعیتی، ایندکس‌های اصلاح غلط املائی و ساختارهایی برای تسریع بازیابی نامحدود top-K است.
- وقتی یک کاربر در مایکروسافت عبارت "Azure security best practices" را جستجو می‌کند، این پرسمان هم مستقیماً به ایندکس‌ها ارسال می‌شود و هم از طریق ماژول تولید کاندیدهای اصلاح غلط املائی عبور می‌کند. این ماژول فقط زمانی فعال می‌شود که پرسمان اصلی نتایج کافی نداشته باشد.

اسناد بازیابی شده به ماژول امتیازدهی ارسال می‌شوند که امتیازات را بر اساس رتبه‌بندی یادگیری ماشین (MLR) محاسبه می‌کند. این تکنیک که بر اساس مفاهیم بخش ۶.۱.۲ ساخته شده و در بخش ۱۵.۴.۱ بیشتر توسعه داده می‌شود، به سیستم اجازه می‌دهد تا با ترکیب صدها سیگنال مختلف، مرتبط ترین نتایج را در صدر نمایش دهد.

در نهایت، این اسناد رتبه‌بندی شده به صورت یک صفحه نتایج به کاربر نمایش داده می‌شوند.

مسیر داده در یک سیستم جستجوی کامل

شکل ۷.۵ یک سیستم جستجوی کامل را نشان می‌دهد که مسیرهای داده عمدتاً برای یک پرسمان متن آزاد را نمایش می‌دهد. این معماری مشابه معماری استفاده شده در سیستم‌های بزرگ مانند جستجوی گوگل یا الکسا (آمازون) است، هرچند در مقیاس بسیار بزرگتر.



► Figure 7.5 A complete search system. Data paths are shown primarily for a free text query.

شکل 7.5 : یک سیستم جست و جو کامل. مسیرهای داده عمدتاً برای یک پرسوجوی متنی آزاد نمایش داده شده اند.

در فصل ۸، بخش ۸.۷، به طور مفصل به تولید خودکار برش‌های متن (snippets) خواهیم پرداخت که به کاربران کمک می‌کند به سرعت بفهمند چرا یک سند با پرسمان آن‌ها مرتبط است.

۷.۳ امتیازدهی فضای برداری و تعامل با عملگرهای پرسمان

مدل فضای برداری را به عنوان یک پارادایم برای پرسمان‌های متن آزاد معرفی کردیم. در این بخش، بررسی می‌کنیم که مدل امتیازدهی فضای برداری چگونه با عملگرهای پرسمانی که در فصل‌های قبلی مطالعه کردیم، ارتباط دارد. این ارتباط باید در دو سطح بررسی شود: از نظر قدرت بیانی پرسمان‌هایی که یک کاربر پیشرفته ممکن است مطرح کند، و از نظر ایندکسی که ارزیابی روش‌های مختلف ارزیابی را پشتیبانی می‌کند.

✦ ارزیابی متن آزاد در موتورهای جستجوی مدرن

در گذشته، پرسمان‌های متن آزاد به این صورت تفسیر می‌شدند که حداقل یکی از واژه‌های پرسمان در هر سند ارزیابی شده وجود داشته باشد. اما امروزه، موتورهای جستجوی وب مانند گوگل این مفهوم را محبوب کرده‌اند که مجموعه‌ای از واژه‌های تایپ شده در جعبه پرسمان (که در ظاهر یک پرسمان متن آزاد است)، معنای یک پرسمان عطفی را دارد که فقط اسنادی را ارزیابی می‌کند که حاوی تمام یا اکثر واژه‌های پرسمان هستند.

برای مثال، وقتی در گوگل عبارت "رستوران ایتالیایی تهران" را جستجو می‌کنید، گوگل به طور ضمنی به دنبال اسنادی می‌گردد که هر سه واژه "رستوران"، "ایتالیایی" و "تهران" را در خود دارند، نه فقط یکی از این واژه‌ها را.

✦ ارزیابی بولی

به وضوح، یک ایندکس فضای برداری می‌تواند برای پاسخ به پرسمان‌های بولی استفاده شود، به شرطی که وزن واژه t در بردار سند d هر زمان که t در d وجود دارد، غیرصفر باشد. برعکس این موضوع صحیح نیست، زیرا یک ایندکس بولی به طور پیش‌فرض اطلاعات وزن واژه را حفظ نمی‌کند.

از دیدگاه کاربر، هیچ راه آسانی برای ترکیب پرسمان‌های فضای برداری و بولی وجود ندارد: پرسمان‌های فضای برداری اساساً یک شکل تجمع شواهد هستند، که در آن وجود واژه‌های بیشتر پرسمان در یک سند به امتیاز آن سند اضافه می‌شود. از سوی دیگر،

بازیابی بولی از کاربر می‌خواهد که یک فرمول برای انتخاب اسناد از طریق وجود (یا عدم وجود) ترکیبات خاصی از کلیدواژه‌ها مشخص کند، بدون اینکه هیچ ترتیب نسبی بین آنها ایجاد کند.

در گذشته، سایت‌هایی مانند AltaVista به کاربران اجازه می‌دادند از عملگرهای بولی مانند AND، OR و NOT استفاده کنند. برای مثال، کاربر می‌توانست پرسمان "java AND NOT coffee" را وارد کند تا اسنادی که حاوی واژه java هستند اما coffee نیستند، بازیابی شوند. امروزه گوگل این عملگرها را به طور مستقیم پشتیبانی نمی‌کند، اما معادل‌هایی مانند استفاده از علامت منها (-) برای NOT را ارائه می‌دهد.

✦ پرسمان‌های حروف جایگزین (Wildcard)

پرسمان‌های wildcard و فضای برداری به ایندکس‌های مختلفی نیاز دارند، به جز در سطح پایه که هر دو می‌توانند با استفاده از postings و یک دیکشنری پیاده‌سازی شوند (مثلاً یک دیکشنری از trigram برای پرسمان‌های wildcard).

اگر یک موتور جستجو به کاربر اجازه دهد تا یک عملگر wildcard را به عنوان بخشی از یک پرسمان متن آزاد مشخص کند (مثلاً پرسمان "نرم* افزار")، ما می‌توانیم بخش wildcard پرسمان را به عنوان ایجاد چندین واژه در فضای برداری تفسیر کنیم (در این مثال، "نرم‌افزار" و "نرم‌افزارسازی" دو واژه چنین هستند) که همگی به بردار پرسمان اضافه می‌شوند.

سایت‌های جستجوی شغلی مانند LinkedIn اغلب از wildcard استفاده می‌کنند. برای مثال، وقتی یک کارفرما "توسعه* وب" را جستجو می‌کند، سیستم ممکن است این را به "توسعه‌دهنده وب"، "توسعه‌دهندگان وب" و "توسعه وب" گسترش دهد و سپس با استفاده از مدل فضای برداری، اسناد را بر اساس تطابق با این واژه‌ها امتیازدهی و رتبه‌بندی کند.

✦ پرسمان‌های عبارتی (Phrase)

نمایش اسناد به صورت برداری اساساً باعث از دست رفتن اطلاعات است: ترتیب نسبی واژه‌ها در یک سند در کدگذاری سند به عنوان یک بردار از بین می‌رود. حتی اگر بخواهیم هر biword را به عنوان یک واژه (و بنابراین یک محور در فضای برداری) در نظر بگیریم، وزن‌ها روی محورهای مختلف مستقل نیستند: برای مثال، عبارت "German shepherd" در محور "german shepherd" کدگذاری می‌شود، اما بلافاصله وزن غیرصفر روی محورهای "german" و "shepherd" نیز دارد.

بنابراین، یک ایندکس که برای بازیابی فضای برداری ساخته شده، به طور کلی نمی‌تواند برای پرسمان‌های عبارتی استفاده شود. علاوه بر این، هیچ راهی برای درخواست امتیاز فضای برداری برای یک پرسمان عبارتی وجود ندارد - ما فقط وزن نسبی هر واژه در یک سند را می‌دانیم.

گوگل به کاربران اجازه می‌دهد با قرار دادن یک عبارت در نقل قول، جستجوی دقیق عبارت را انجام دهند. برای مثال، جستجوی "هوش مصنوعی" فقط اسنادی را بازیابی می‌کند که این عبارت دقیق به همان ترتیب در آنها وجود دارد. این با جستجوی هوش مصنوعی (بدون نقل قول) متفاوت است که از مدل فضای برداری استفاده می‌کند و اسنادی را بر اساس وجود واژه‌های "هوش" و "مصنوعی" رتبه‌بندی می‌کند، حتی اگر این واژه‌ها کنار هم نباشند.

در حالی که این دو پارادایم بازیابی (عبارتی و فضای برداری) در نتیجه پیاده‌سازی‌های مختلفی از نظر ایندکس‌ها و الگوریتم‌های بازیابی دارند، آنها در برخی موارد می‌توانند به صورت مفیدی ترکیب شوند، مانند مثال سه مرحله‌ای تجزیه پرسمان در بخش ۷.۲.۳.

❁ جمع‌بندی فصل ۷: محاسبه امتیاز در سیستم جستجوی کامل

در این فصل با روش‌های بهینه‌سازی برای محاسبه امتیاز و ساخت یک سیستم جستجوی کامل آشنا شدیم:

- الگوریتم FASTCOSINESCORE را برای محاسبه سریع امتیازات برداری معرفی کردیم. این همان روشی است که به گوگل اجازه می‌دهد میلیاردها سند را در کسری از ثانیه رتبه‌بندی کند.
- روش‌های بازیابی تقریبی K سند برتر را بررسی کردیم. این تکنیک‌ها به سیستم‌های جستجو کمک می‌کنند تا با هزینه کمتر، نتایج مرتبط را سریع‌تر پیدا کنند.
- فهرست‌های چندلایه (Tiered indexes) را به‌عنوان راهی برای بهینه‌سازی جستجو معرفی کردیم. این روش در سیستم‌های بزرگ مانند گوگل استفاده می‌شود تا نتایج را به سرعت نمایش دهد.
- امتیازدهی بر اساس نزدیکی واژه‌ها (Query-term proximity) را بررسی کردیم. این همان روشی است که باعث می‌شود وقتی «پیتزا نزدیک من» را جستجو می‌کنید، نتایجی نمایش داده شود که این دو کلمه در کنار هم قرار دارند.
- طراحی توابع تحلیل و امتیازدهی را برای ترکیب عوامل مختلف در رتبه‌بندی بررسی کردیم. این همان روشی است که به گوگل اجازه می‌دهد صدها عامل مختلف را برای رتبه‌بندی نتایج ترکیب کند.

بیا یاد این مفاهیم را با چند مثال واقعی مقایسه کنیم:

- ♦ **بازیابی تقریبی K سند برتر:** تصور کنید در دیجی‌کالا به دنبال «گوشی سامسونگ با حافظه ۲۵۶ گیگابایت» می‌گردید. دیجی‌کالا از روش‌های بازیابی تقریبی استفاده می‌کند تا به جای بررسی تمام محصولات، فقط تعداد محدودی از محصولات مرتبط را بررسی کند. این کار باعث می‌شود نتایج سریع‌تر نمایش داده شوند، حتی اگر ممکن است چند محصول مرتبط از قلم بیفتند. این روش در سیستم‌های بزرگ با میلیون‌ها محصول ضروری است.
- ♦ **فهرست‌های چندلایه (Tiered indexes):** گوگل از فهرست‌های چندلایه برای جستجو استفاده می‌کند. تصور کنید در گوگل «رستوران ایتالیایی در تهران» را جستجو می‌کنید. گوگل ابتدا در لایه اول فقط رستوران‌هایی را بررسی می‌کند که امتیاز بالایی دارند و کلمات کلیدی اصلی را در عنوان خود دارند. اگر نتایج کافی نبود، به لایه‌های بعدی می‌رود و رستوران‌های بیشتری را بررسی می‌کند. این روش باعث می‌شود نتایج مرتبط سریع‌تر نمایش داده شوند.
- ♦ **امتیازدهی بر اساس نزدیکی واژه‌ها:** وقتی در گوگل «پزشک متخصص قلب در شیراز» را جستجو می‌کنید، گوگل به اسنادی امتیاز بالاتری می‌دهد که این واژه‌ها در کنار هم قرار دارند. برای مثال، صفحه‌ای که عبارت «پزشک متخصص قلب در شیراز» را دارد، امتیاز بالاتری از صفحه‌ای می‌گیرد که این واژه‌ها در نقاط مختلف صفحه پراکنده شده‌اند. این روش باعث می‌شود نتایج مرتبط‌تر در بالای لیست قرار گیرند.
- ♦ **طراحی توابع تحلیل و امتیازدهی:** گوگل از صدها عامل برای رتبه‌بندی نتایج استفاده می‌کند: از امتیازدهی برداری گرفته تا کیفیت صفحه، سرعت بارگذاری، تجربه کاربری و ده‌ها عامل دیگر. این عوامل با استفاده از یادگیری ماشین ترکیب می‌شوند تا بهترین نتایج را برای شما نمایش دهد. برای مثال، وقتی «خرید آنلاین کتاب» را جستجو می‌کنید، گوگل فروشگاه‌های معتبر با امتیاز بالا، سرعت بارگذاری خوب و نظرات مثبت کاربران را در بالای نتایج نمایش می‌دهد.

این فصل مفاهیم اساسی ساخت یک سیستم جست‌وجوی کامل را فراهم می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:

- یادگیری ماشین برای بهبود رتبه‌بندی - روش‌هایی که گوگل از آن‌ها برای یادگیری از رفتار کاربران و بهبود نتایج جستجو استفاده می‌کند
- ارزیابی سیستم‌ها (دقت، بازیابی، F1) - معیارهایی برای سنجش اینکه آیا موتور جستجوی ما واقعاً خوب کار می‌کند یا نه
- مدل‌های احتمالی - تکنیک‌هایی که به سیستم‌های جستجو اجازه می‌دهند احتمال مرتبط بودن یک سند را تخمین بزنند
- تکنیک‌های پیشرفته بازیابی اطلاعات - روش‌هایی که به سیستم‌های جستجو اجازه می‌دهند مفاهیم پیچیده را درک کنند

Hamid Namjoo, Amirhosein Hemmati, Ali mojahed
Creative Commons Attribution 4.0 International (CC BY 4.0)
<https://github.com/hrnrxb/IR-Textbook>